Pontificia Universidad Católica del Perú
Facultad de Ciencias e Ingeniería
Departamento Académico de Ingeniería



1INF02 APRENDIZAJE DE MÁQUINA

Perceptrón y Máquinas de Vectores de Soporte (SVM)

Prof. Rodrigo Uribe

Organización de las Clases

(10 min) Preguntas y consultas.

(30 min) Respuesta *Vibe Quest.*

(45 min) Perceptrón y SVM.



(10 min) Break

(25 min) Errores en SVM y el truco del Kernel.

(30 min) Ejercicios matemáticos.

(10 min) Preguntas *Vibe Quest.*

Respuestas Vibe Quest

- La solución directa de regresión lineal usa $(X^TX)^{-1}X^Tt$, mientras que el descenso de gradiente es iterativo. Crea una simulación que muestre ambos métodos llegando al mismo resultado. ¿A partir de cuántas dimensiones (D) el descenso de gradiente se vuelve más eficiente que invertir la matriz?
- Crea un “laboratorio de mapeo de características” donde puedas probar diferentes bases: polinomiales, trigonométricas (senos/cosenos), y gaussianas. ¿Cómo afecta la elección de la base a la forma de la función ajustada? Encuentra una combinación que capture patrones periódicos en datos sintéticos
- La función sigmoide $\sigma(z) = 1/(1+e^{-z})$ transforma valores reales a “probabilidades”. Construye un visualizador interactivo que muestre cómo cambia la frontera de decisión de un clasificador logístico mientras ajustas los pesos w y el sesgo b. ¿Cómo se relaciona la magnitud de w con la 'confianza' del clasificador? Utiliza un análisis bayesiano para responder.
- Softmax generaliza la sigmoide a múltiples clases, convirtiendo scores arbitrarios en probabilidades que suman 1. ¿Cómo funciona esta 'competencia' entre clases?¿Se podría considerar probabilidad o no? Utiliza un análisis bayesiano para responder.

Clase 1:

¿Qué es aprendizaje de máquina?

(Computadoras aprendiendo de datos automáticamente)

Clase 2:

¿Cómo se toman decisiones (modelado) con datos?

Clase 3:

¿Cómo cambio el análisis y modelado de datos?

Clase 4:

¿Cómo los modelos aprenden de equivocarse?

Clase 5:

¿Cómo encontramos la mejor frontera de decisión?

Encuentra la mejor explicación...

En un experimento de caída libre, se midieron los tiempos de caída de una bola desde diferentes alturas. Los tiempos registrados son consistentemente más largos que los predichos por la teoría de la gravedad.



Posibles explicaciones

a) La teoría de la gravedad es incorrecta.
b) Una fuerza adicional (como resistencia del aire) afecta la caída.
c) El experimento se hizo en un lugar con gravedad diferente (¿la Luna?).
d) El cronómetro estaba descalibrado.

La Navaja de Ockham

La navaja de Ockham es un principio que sugiere que, entre varias explicaciones posibles, la más simple suele ser la correcta.




La navaja de
Ockham
explicada por Lisa
Simpson



Motivación: Regularización

- **La navaja de Occam:** prefiera la hipótesis más sencilla

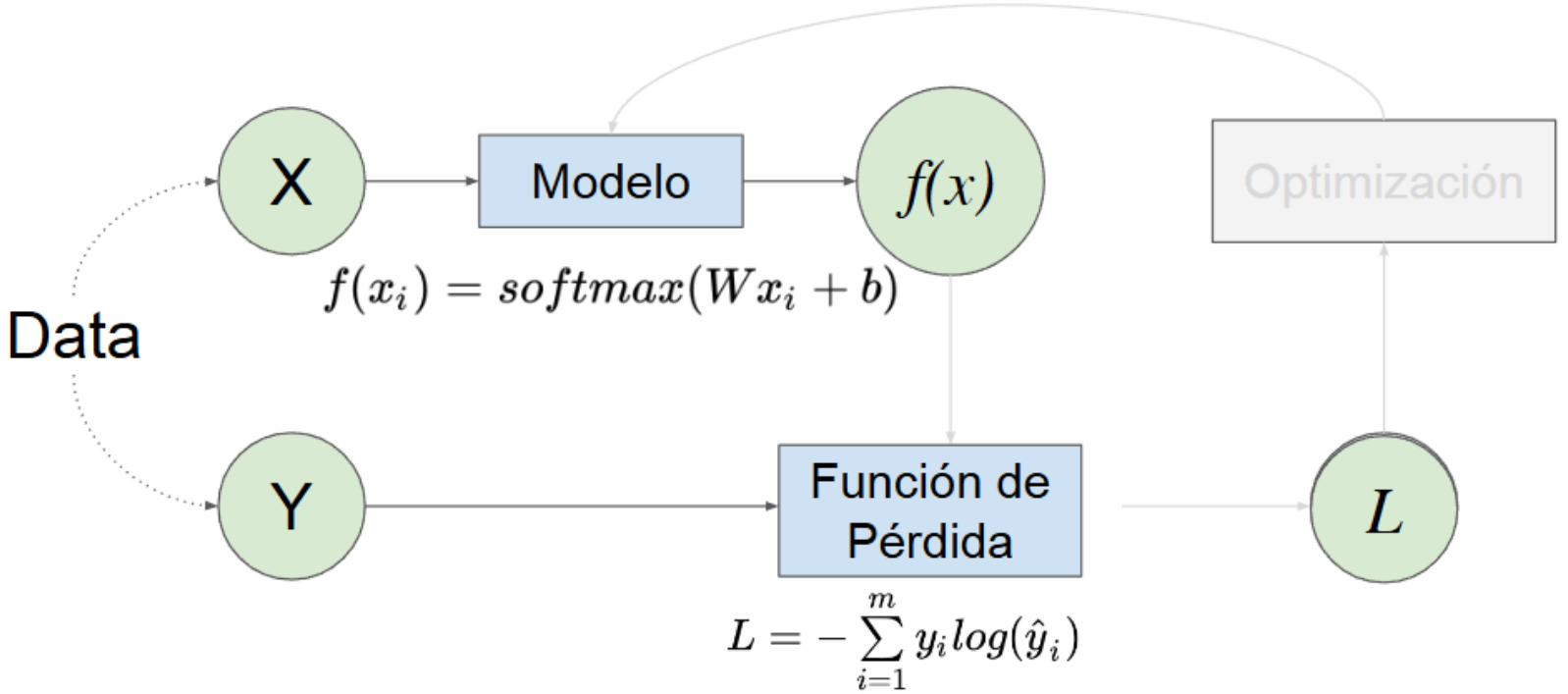
- ¿Qué significa que una hipótesis (o modelo) sea **simple**?
  1. número reducido de características (**selección del modelo**)
  2. pequeño número de características "importantes" (**contracción**)

Una vez reducimos lo “esencial” del problema…

¿Qué modelo utilizamos?

Recordando…

Regresión logística



¿Cuál era el problema de la regresión logística?

Recordando…

Funciones de regularización

hiper-parámetro
de regularización

$$L + \lambda R(W)$$

L2 regularization $R(W) = \sum_k \sum_l W_{k,l}^2$

L1 regularization $R(W) = \sum_k \sum_l |W_{k,l}|$

Elastic net (L1 + L2)

Max norm regularization (n$R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$

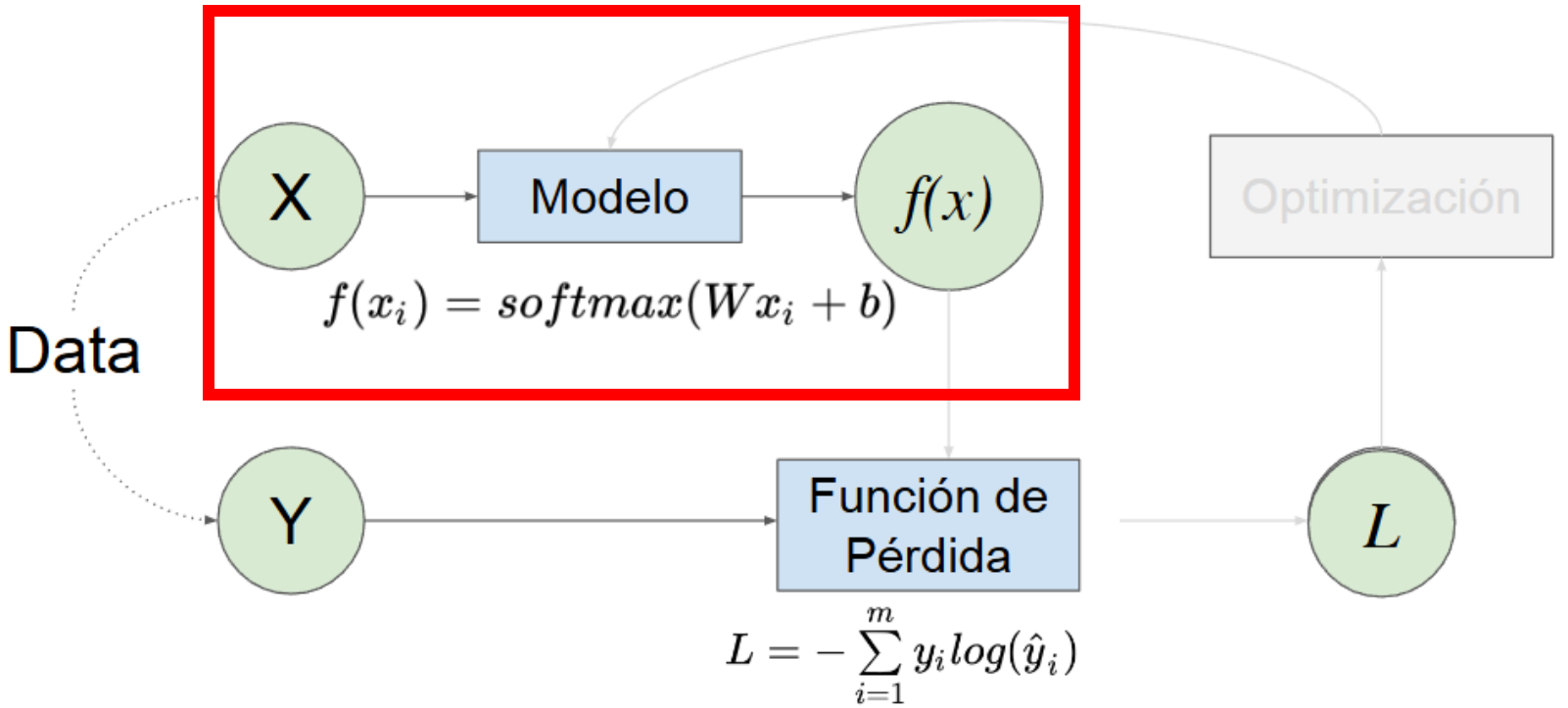
Dropout (will see later)

Investigar sobre estas funciones!!!

??

“Simplifiquemos” la formulación

Regresión logística



¿Probabilidad?

Definición del modelo: **Clasificador Lineal**

sepal_length	sepal_width	petal_length	petal_width
4.6	3.6	1.0	0.2

Variables independientes:
[4x1]

data weights

$$\xrightarrow{f(\mathbf{x},\mathbf{W})} \quad f(x, W) = Wx$$

setosa!

3 resultados, indicando la probabilidad de cada clase:

[1, 0, 0]
Setosa Versicolor Virginica
(one hot encoding)

[3x1]

¿Qué tenían en común la regresión lineal y la logística?



Comparación de Actualizaciones

- Comparación de actualizaciones de descenso de gradiente:
- Regresión lineal:

$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\alpha}{N} \sum_{i=1}^{N} (y^{(i)} - t^{(i)})\, \mathbf{x}^{(i)}$$

- Regresión logística:

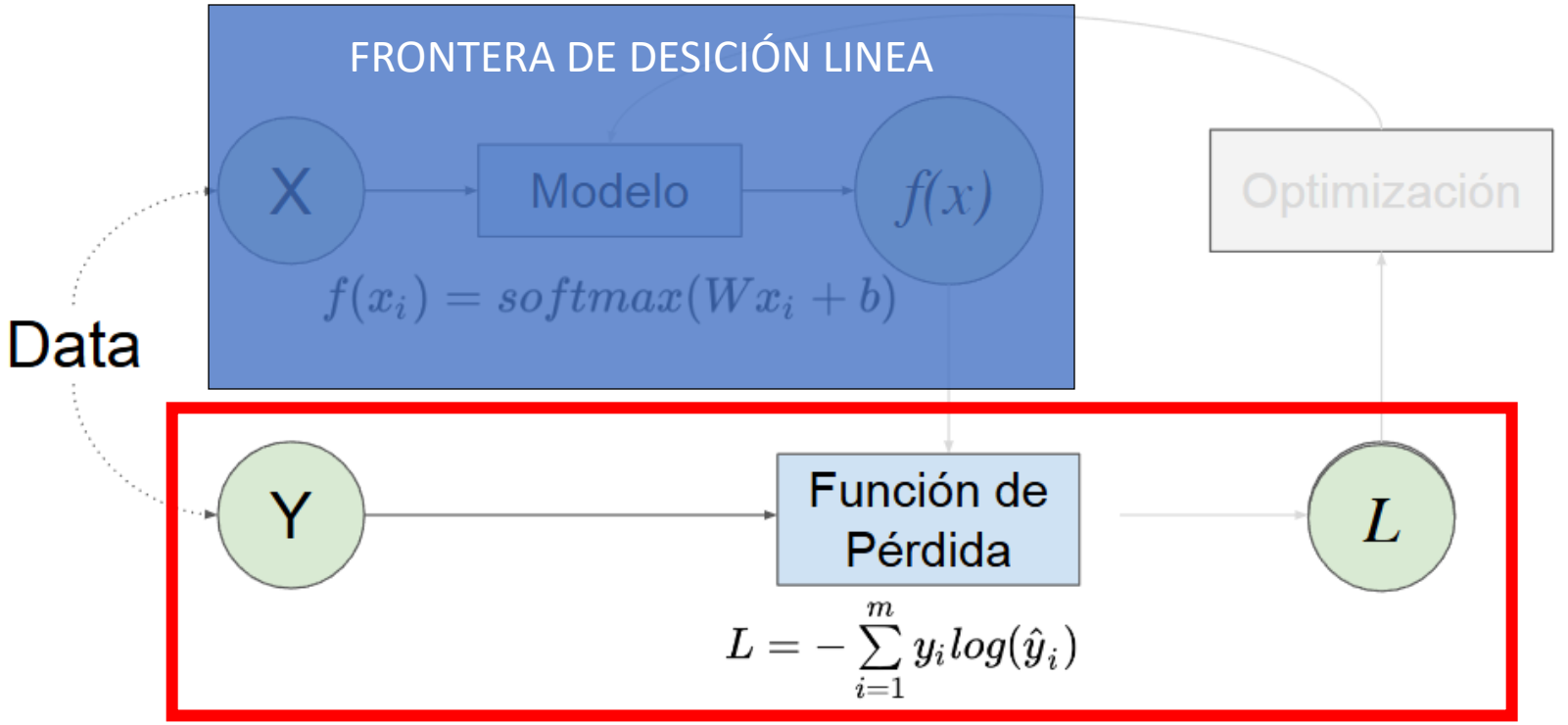
$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\alpha}{N} \sum_{i=1}^{N} (y^{(i)} - t^{(i)})\, \mathbf{x}^{(i)}$$

- ¡No es coincidencia! Ambos son ejemplos de **modelos lineales generalizados**.

¿Se puede generalizar(simplificar) esta idea?

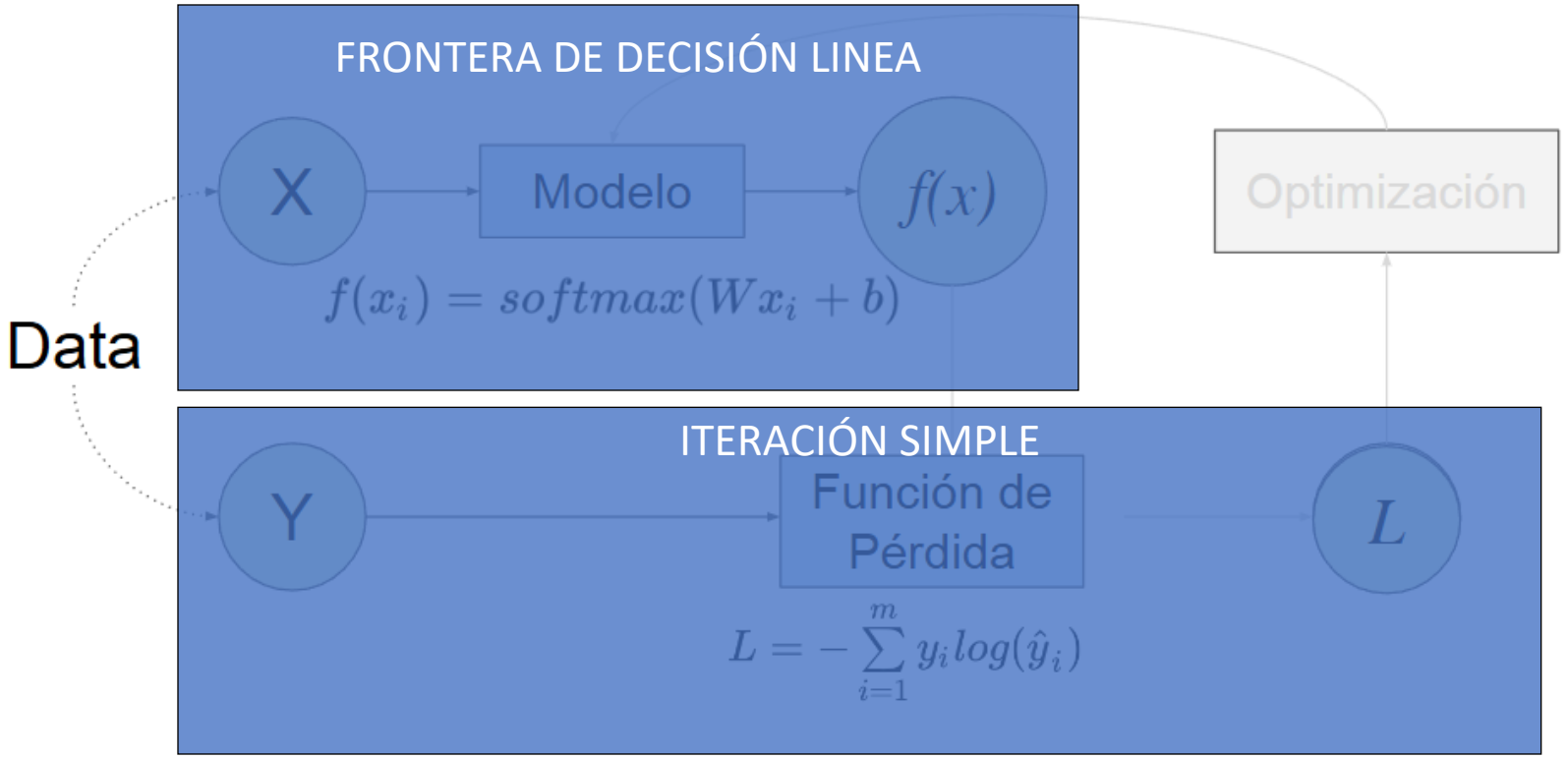
“Simplifiquemos” la formulación

Regresión logística



“Simplifiquemos” la formulación

~~Regresión logística~~ **Perceptrón**

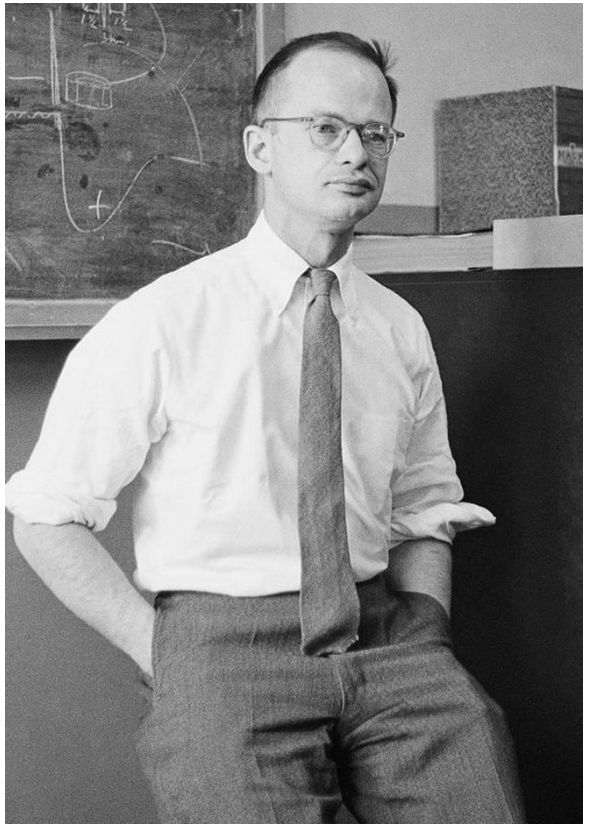


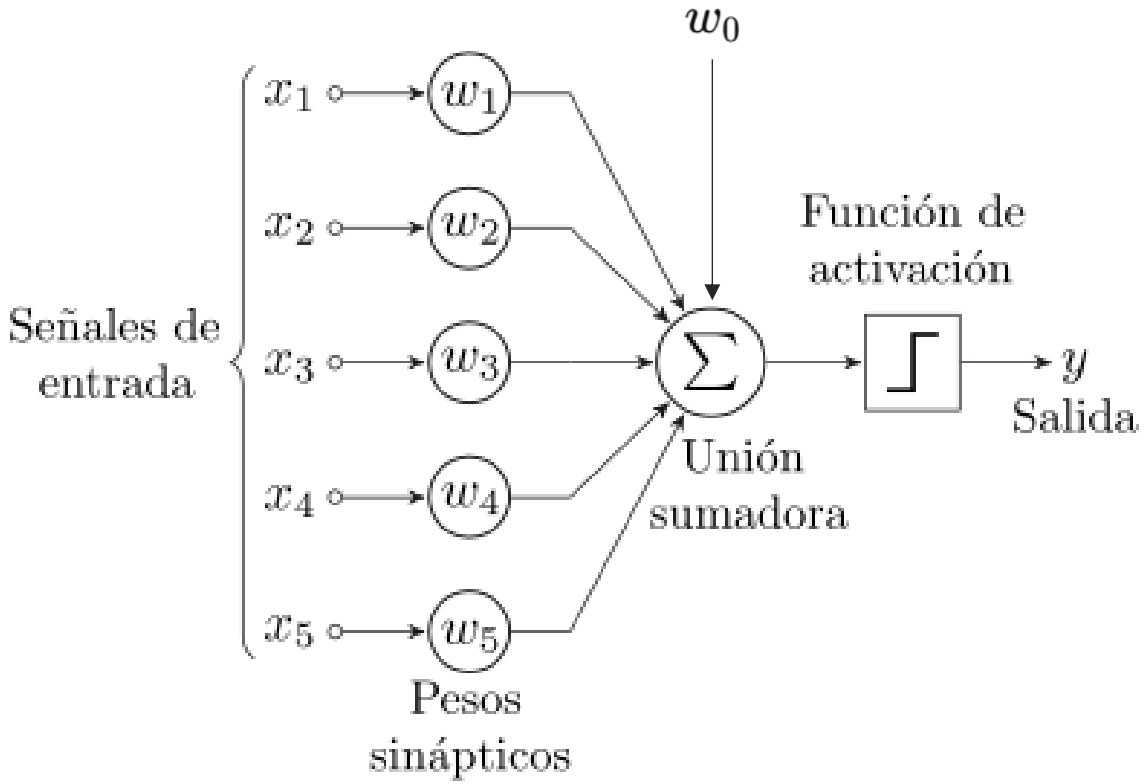


https://www.youtube.com/watch?v=cNxadbrN_aI

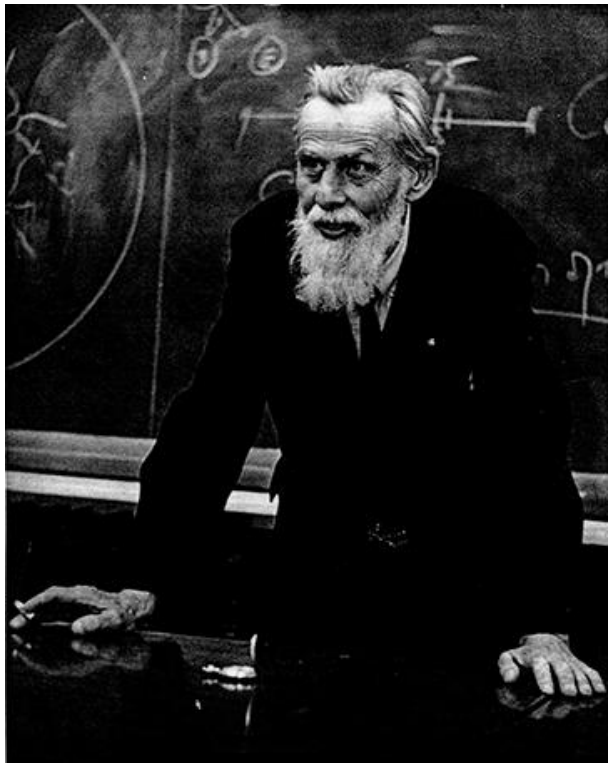
El Perceptrón (1943)

Walter Pitts

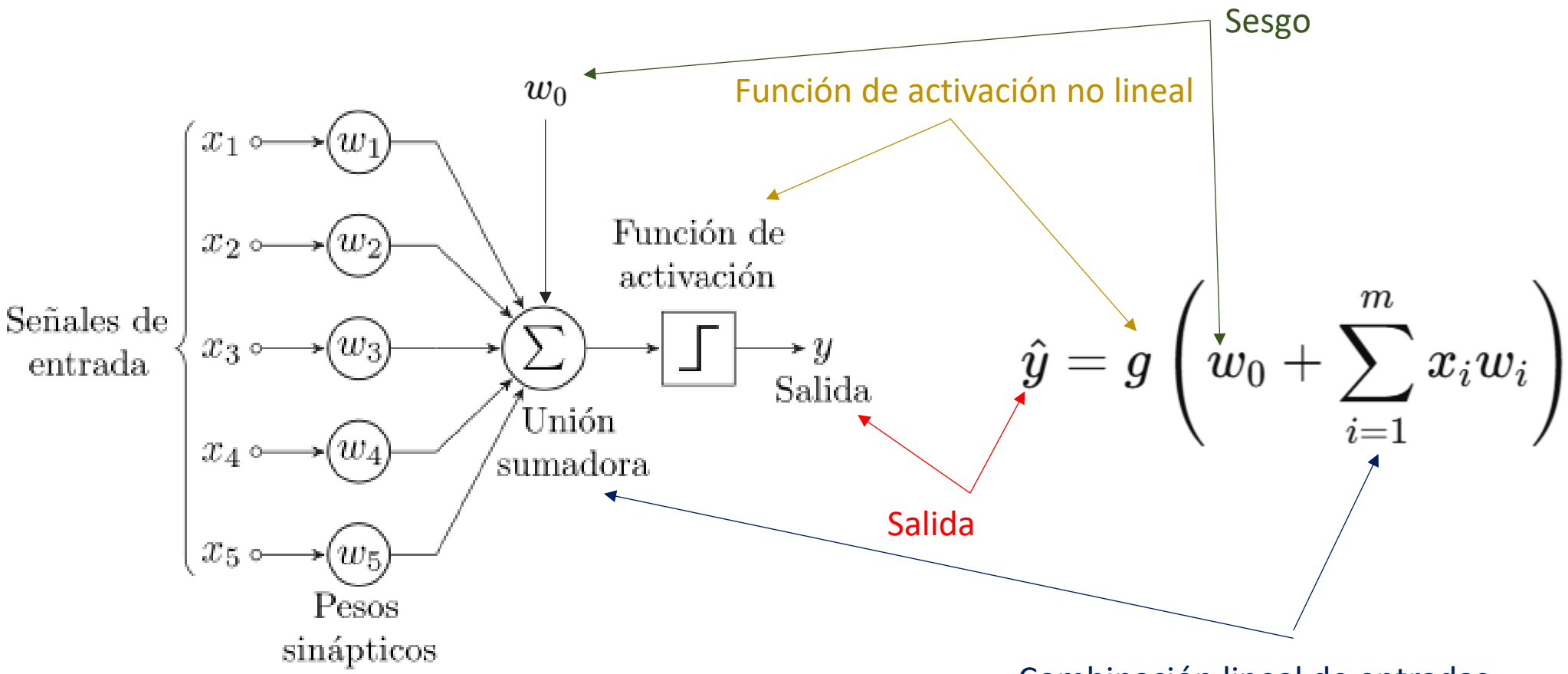




Warren McCulloch



El Perceptrón (1943)



- Se puede ver la regresión logística como una sola unidad.
- Una vez que conectamos muchas unidades en una red, se vuelven mucho más poderosas.

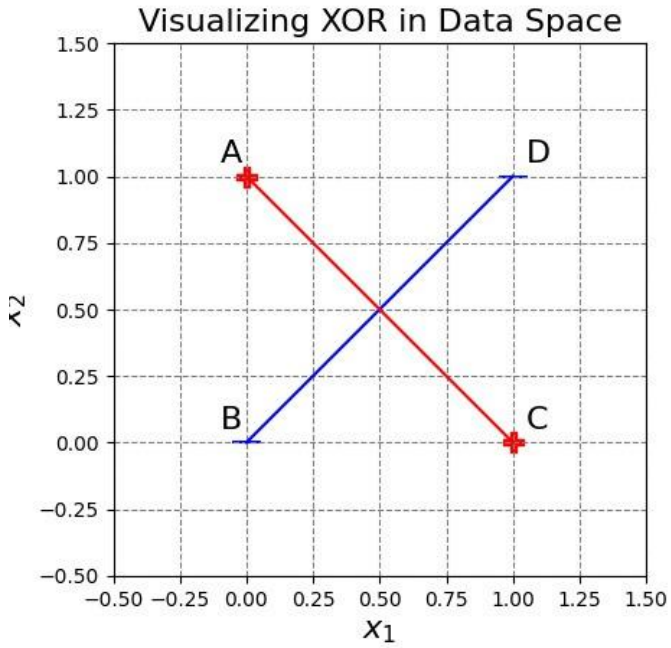
El Perceptrón como una generalización simplificada de clasificación de modelos lineales

	Logistic Regression	Softmax Regression
Linear Model	$z = \mathbf{w}^T\mathbf{x}$	$\mathbf{z} = \mathbf{W}\mathbf{x}$
Activation	$y = \sigma(z) = \frac{1}{1+e^{-z}}$	$\mathbf{y} = \text{softmax}(\mathbf{z})$ $y_k = \frac{e^{z_k}}{\sum_{m=1}^{K} e^{z_m}}$
Cross-Entropy Loss	$L_{CE}(y,t)$ $= -t \log y$ $-(1-t)\log(1-y)$	$L_{CE}(\mathbf{y},\mathbf{t}) = -\mathbf{t}^T \log(\mathbf{y})$ $= -\sum_{k=1}^{K} t_k \log(y_k)$

XOR NO es linealmente separable

Las neuronas individuales son clasificadores lineales y tienen un poder expresivo limitado. Por ejemplo, la función XOR no es linealmente separable (se puede demostrar mediante convexidad).

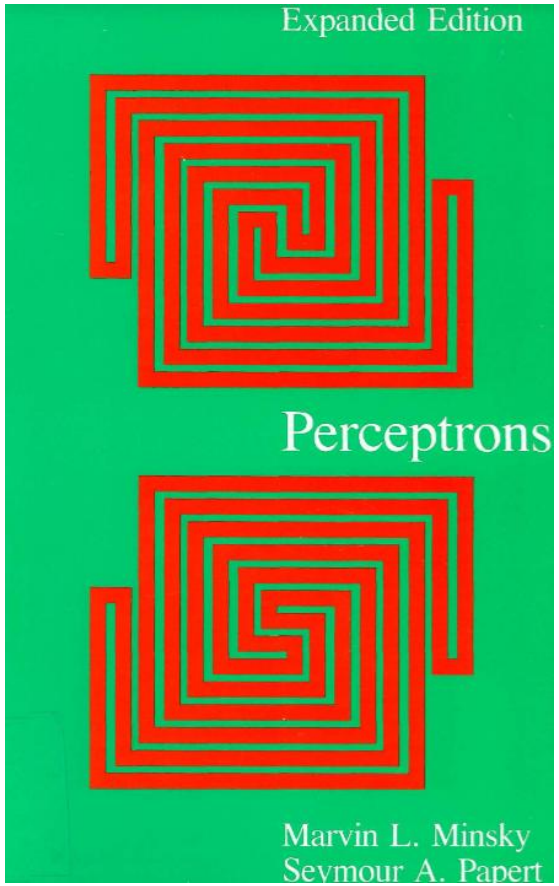
	x_1	x_2	t
A	0	1	1
B	0	0	0
C	1	0	1
D	1	1	0



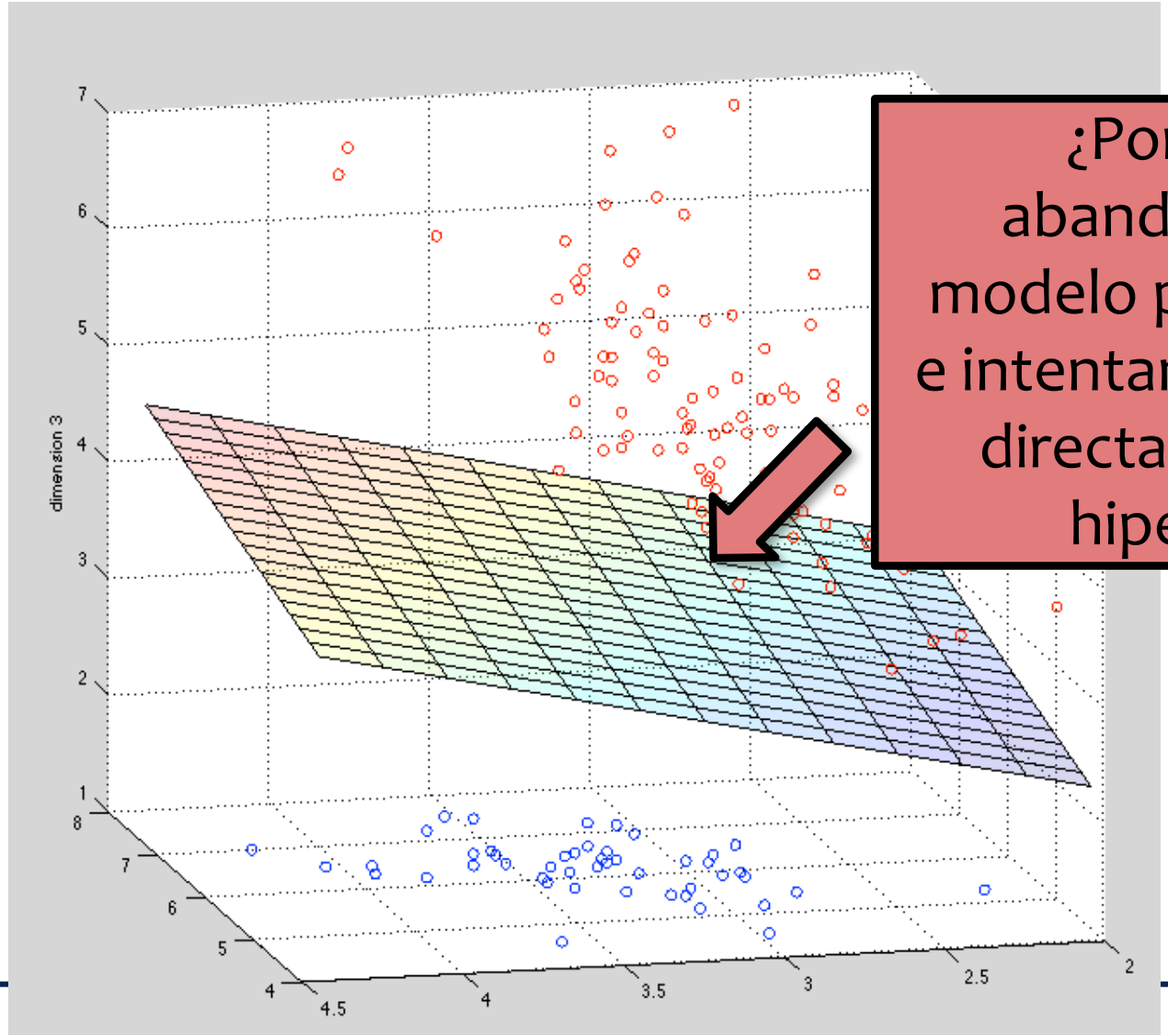
¿Qué truco podemos usar?

Primera Invierno de la IA: Limitaciones del Perceptrón

- En 1969, Minsky y Papert demostraron que el perceptrón no podía resolver problemas no lineales como XOR.
- Esto expuso graves limitaciones en las redes neuronales de una sola capa.
- Causó una pérdida de interés y financiamiento, iniciando el primer invierno de la inteligencia artificial.
- A pesar de ello era el modelo mas “general” que se conocía.

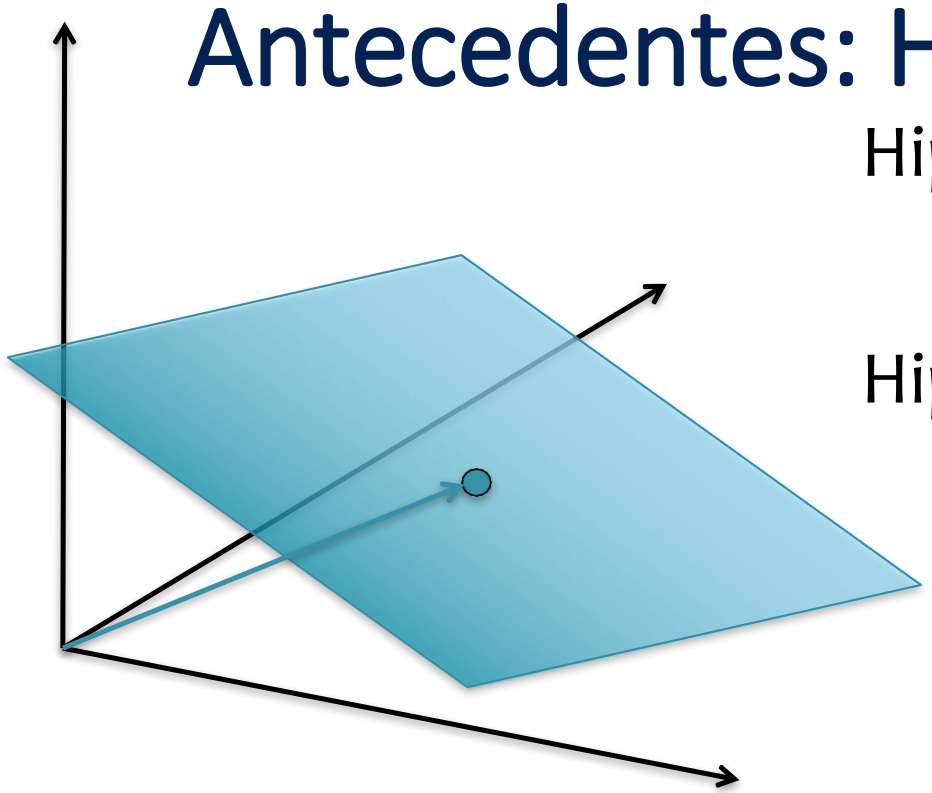


¿Cómo aprendía el perceptrón?



¿Por qué no abandonamos el modelo probabilistico e intentamos aprender directamente este hiperplano?

Antecedentes: Hiperplanos



Hiperplano (Definición 1):

$$\mathcal{H} = \{\mathbf{x} : \mathbf{w}^T\mathbf{x} = b\}$$

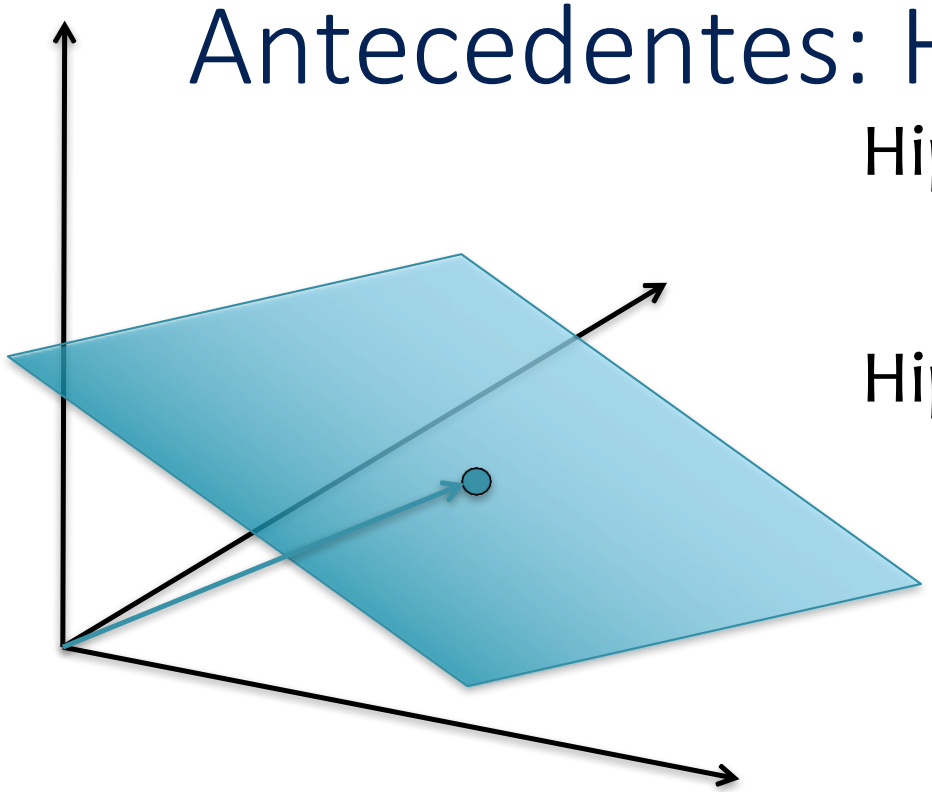
Hiperplano (definición 2):

$$\mathcal{H} = \{\mathbf{x} : \mathbf{w}^T\mathbf{x} = 0 \text{ and } \mathrm{x}_0 = 1\}$$

$$\mathcal{H}^+ = \{\mathbf{x} : \mathbf{w}^T\mathbf{x} > 0 \text{ and } \mathrm{x}_0 = 1\}$$

$$\mathcal{H}^- = \{\mathbf{x} : \mathbf{w}^T\mathbf{x} < 0 \text{ and } \mathrm{x}_0 = 1\}$$

Antecedentes: Hiperplanos



Hiperplano (Definición 1):

$\mathcal{H}$

Hiperpla

$\mathcal{H}$

Para modelar directamente el hiperplano se utilizaría una función de decisión:

$h(\mathbf{t}) = \text{sign}(\boldsymbol{w}^T \, \mathrm{t})$

para: $y \in \{-1, +1\}$

$$\mathcal{H}^+ = \{\mathbf{x} : \mathbf{w}^T\mathbf{x} > 0 \text{ and } \mathrm{x}_0 = 1\}$$

$$\mathcal{H}^- = \{\mathbf{x} : \mathbf{w}^T\mathbf{x} < 0 \text{ and } \mathrm{x}_0 = 1\}$$

¿Cómo aprendía el perceptrón?

Para i = 1, 2, 3, **…:**

- **Recibir** una instancia no etiquetada $\mathbf{x}^{(i)}$
- **Predecir** $y' = h(\mathbf{x})^{(i)}$
- **Recibir** etiqueta verdadera $y^{(i)}$
  Compruebe si es correcto $(y' == y)^{(i)}$

Objetivo:

- **Minimizar** el número de **errores**

Algoritmo del perceptrón

- **Datos:** Las entradas son vectores continuos de longitud K. Las salidas son discretas.

$$(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \ldots$$

  - donde $\mathbf{t} \in \mathbb{R}^K$ e $y \in \{+1, -1\}$

- **Predicción:** Salida determinada por el hiperplano.

$$\hat{y} = h_{\boldsymbol{\theta}}(\mathbf{x}) = \text{sign}(\boldsymbol{w}^T \mathbf{x})$$

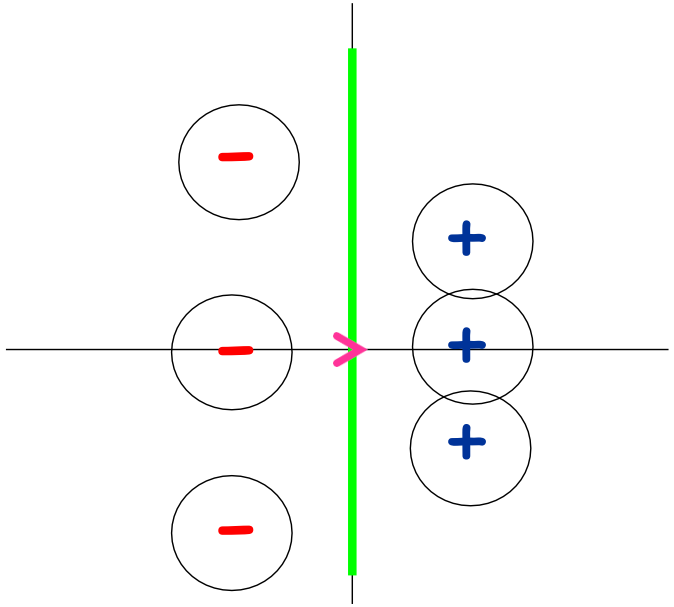
$$\text{sign}(a) = \begin{cases} 1, & \text{si } a \geq 0 \\ -1, & \text{en caso contrario} \end{cases}$$

Aprendizaje: Procedimiento iterativo:

- **mientras** no converja
  - **reciba** el siguiente ejemplo $(\mathbf{x}^{(i)}, y)^{(i)}$
  - **predecir** $y' = h(\mathbf{x})^{(i)}$
  - **si** error positivo: **añada** $\mathbf{x}^{(i)}$ a los parámetros
  - **si** error negativo: **reste** $\mathbf{x}^{(i)}$ de los parámetros

Algoritmo del perceptrón: Ejemplo

Ejemplo:



Algoritmo:

- Establezca t=1, comience con el vector de pesos todo-cero w_1
- Dado el ejemplo x, prediga positivo si $\theta_t \cdot x \geq 0$.
- En caso de error, actualice como sigue:
  - Error en el positivo, actualice $\theta_{t+1} \rightarrow \theta_t + x$
  - Error en el negativo, actualice $\theta_{t+1} \rightarrow \theta_t - x$

$\theta_1 = (0,0)$

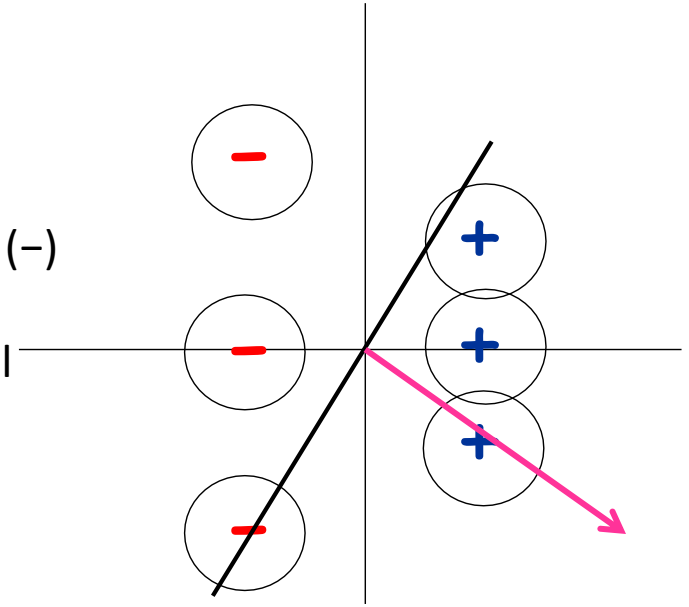
Empezamos con **θ_1 = (0, 0)**

Algoritmo del perceptrón: Ejemplo

Ejemplo:

(-1,2) es (-) X

Evaluamos el punto (-1, 2) que es negativo (–)
$\theta_1 \cdot x = (0,0) \cdot (-1,2) = 0 \cdot (-1) + 0 \cdot 2 = \mathbf{0}$
Como 0 ≥ 0, predecimos **positivo**, pero el punto es **negativo**.



Algoritmo:

- Establezca t=1, comience con el vector de pesos todo-cero w_1.
- Dado el ejemplo x, prediga positivo si $\theta_t \cdot x \geq 0$.
- En caso de error, actualice como sigue:
  - Error en el positivo, actualice $\theta_{t+1} \rightarrow \theta_t + x$
  - Error en el negativo, actualice $\theta_{t+1} \rightarrow \theta_t - x$

$\theta_1 = (0,0)$

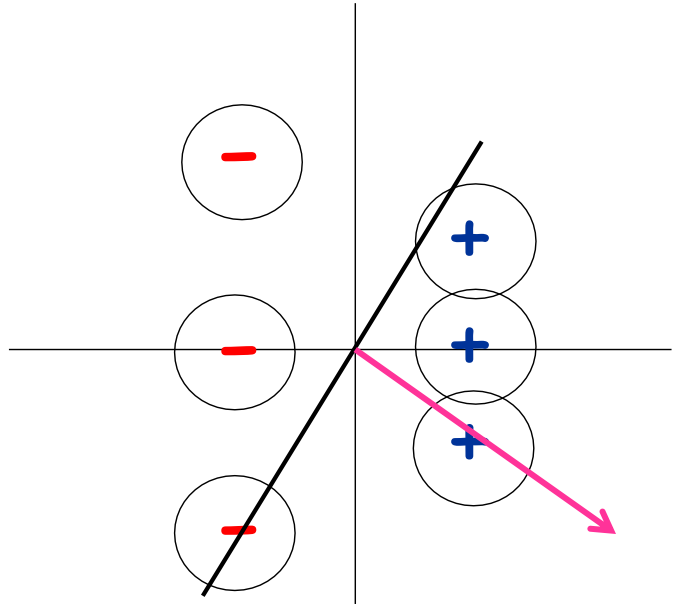
$\theta_2 = \theta_1 - (-1,2) = (1, -2)$

Algoritmo del perceptrón: Ejemplo

Ejemplo:

(-1,2) es (-) X (no cumple)

(1,0) es (+) ✔ (cumple)



Algoritmo:

- Establezca t=1, comience con el vector de pesos todo-cero $w_{.1}$
- Dado el ejemplo x, prediga positivo si $\theta_t \cdot x \geq 0$.
- En caso de error, actualice como sigue:
  - Error en el positivo, actualice $\theta_{t+1} \rightarrow \theta_t + x$
  - Error en el negativo, actualice $\theta_{t+1} \rightarrow \theta_t - x$

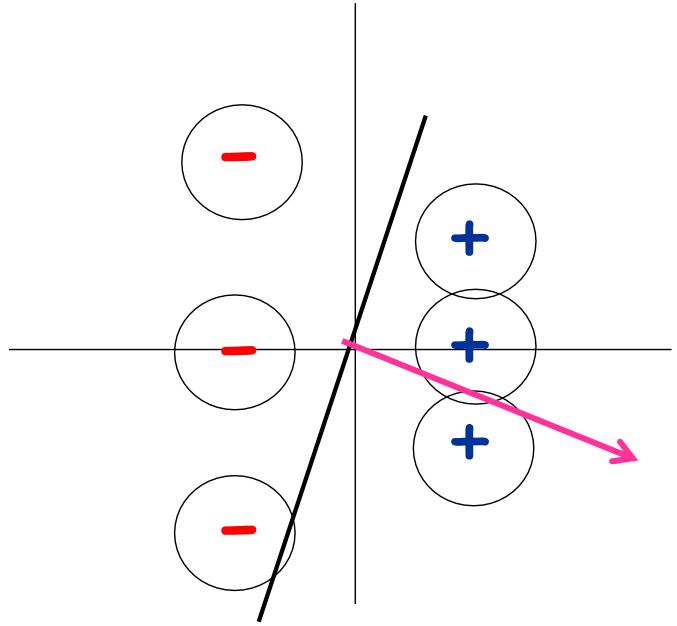
$\theta_1 = (0,0)$

$\theta_2 = \theta_1 - (-1,2) = (1, -2)$

Algoritmo del perceptrón: Ejemplo

Ejemplo:

(-1,2) es (-) X (no cumple)
(1,0) es (+) ✔ (cumple)
(1,1) es (+) X (no cumple)



Algoritmo:

- Establezca t=1, comience con el vector de pesos todo-cero $w_{.1}$
- Dado el ejemplo x, prediga positivo si $\theta_t \cdot x \geq 0$.
- En caso de error, actualice como sigue:
  - Error en el positivo, actualice $\theta_{t+1} \rightarrow \theta_t + x$
  - Error en el negativo, actualice $\theta_{t+1} \rightarrow \theta_t - x$

$\theta_1 = (0,0)$

$\theta_2 = \theta_1 - (-1,2) = (1, -2)$

$\theta_3 = \theta_2 + (1,1) = (2, -1)$

Algoritmo del perceptrón: Ejemplo

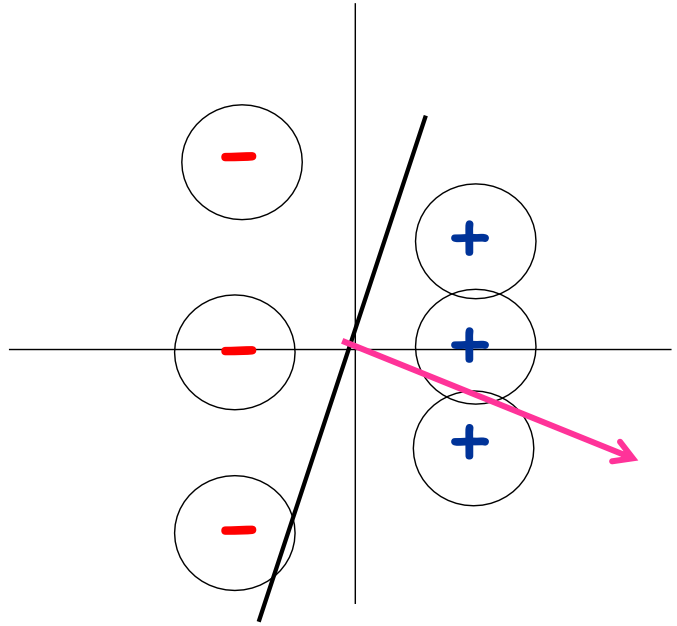
Ejemplo:

(-1,2) es (-) X (no cumple)

(1,0) es (+) ✔ (cumple)

(1,1) es (+) X (no cumple)

(1,0) es (-) ✔ (cumple)



Algoritmo:

- Establezca t=1, comience con el vector de pesos todo-cero $w_{.1}$
- Dado el ejemplo x, prediga positivo si $\theta_t \cdot x \geq 0$.
- En caso de error, actualice como sigue:
  - Error en el positivo, actualice $\theta_{t+1} \rightarrow \theta_t + x$
  - Error en el negativo, actualice $\theta_{t+1} \rightarrow \theta_t - x$

$\theta_1 = (0,0)$

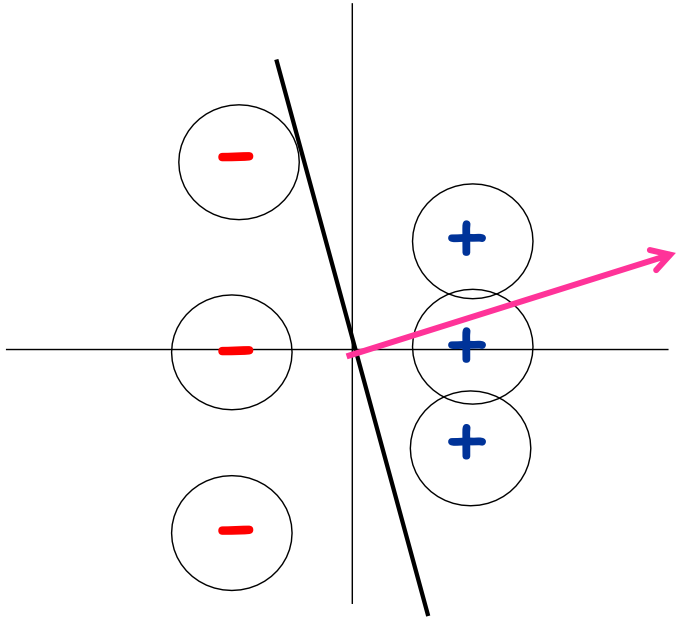
$\theta_2 = \theta_1 - (-1,2) = (1, -2)$

$\theta_3 = \theta_2 + (1,1) = (2, -1)$

Algoritmo del perceptrón: Ejemplo

Ejemplo:

(-1,2) es (-) X (no cumple)
(1,0) es (+) ✔ (cumple)
(1,1) es (+) X (no cumple)
(1,0) es (-) ✔ (cumple)
(-1,-2) es (-) X (no cumple)
(1,-1) es (+) ✔ (cumple)



Algoritmo:

- Establezca t=1, comience con el vector de pesos todo-cero w_1.
- Dado el ejemplo x, prediga positivo si $\theta_t \cdot x \geq 0$.
- En caso de error, actualice como sigue:
  - Error en el positivo, actualice $\theta_{t+1} \rightarrow \theta_t + x$
  - Error en el negativo, actualice $\theta_{t+1} \rightarrow \theta_t - x$

$_1\ \theta = (0,0)$

$_2\ \theta = \theta_1 - (-1,2) = (1,-2)$

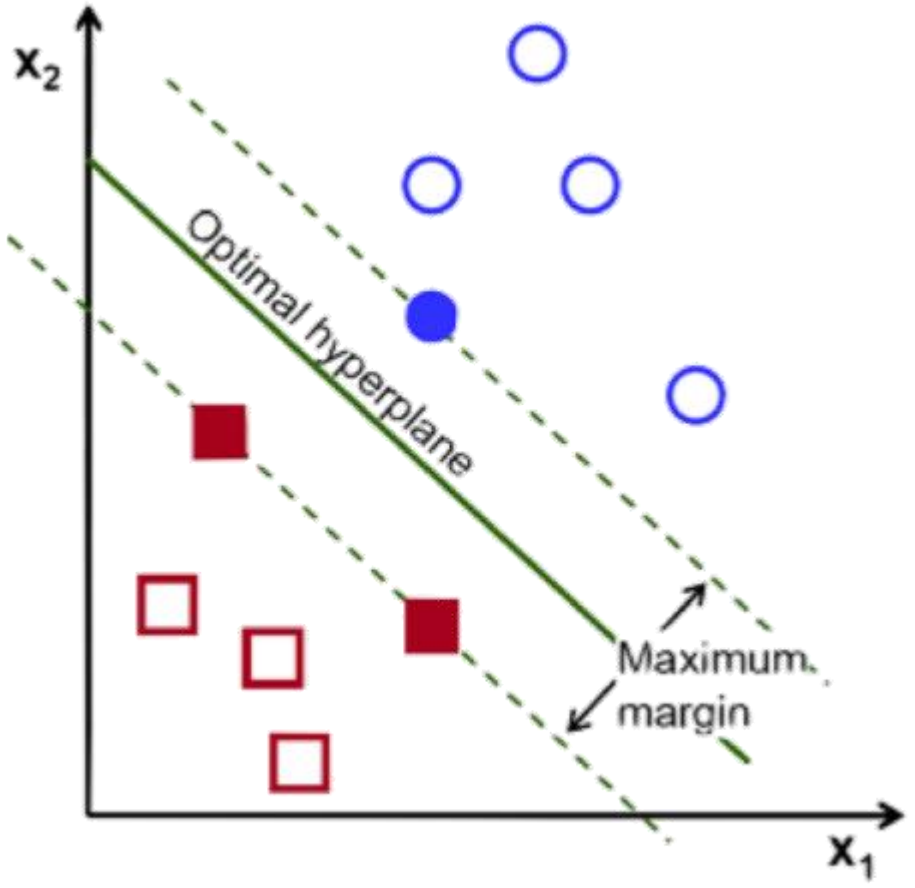
$_3\ \theta = \theta_2 + (1,1) = (2,-1)$

$_4\ \theta = \theta_3 - (-1,-2) = (3,1)$

¿Qué problemas encuentran en esta simplificación?

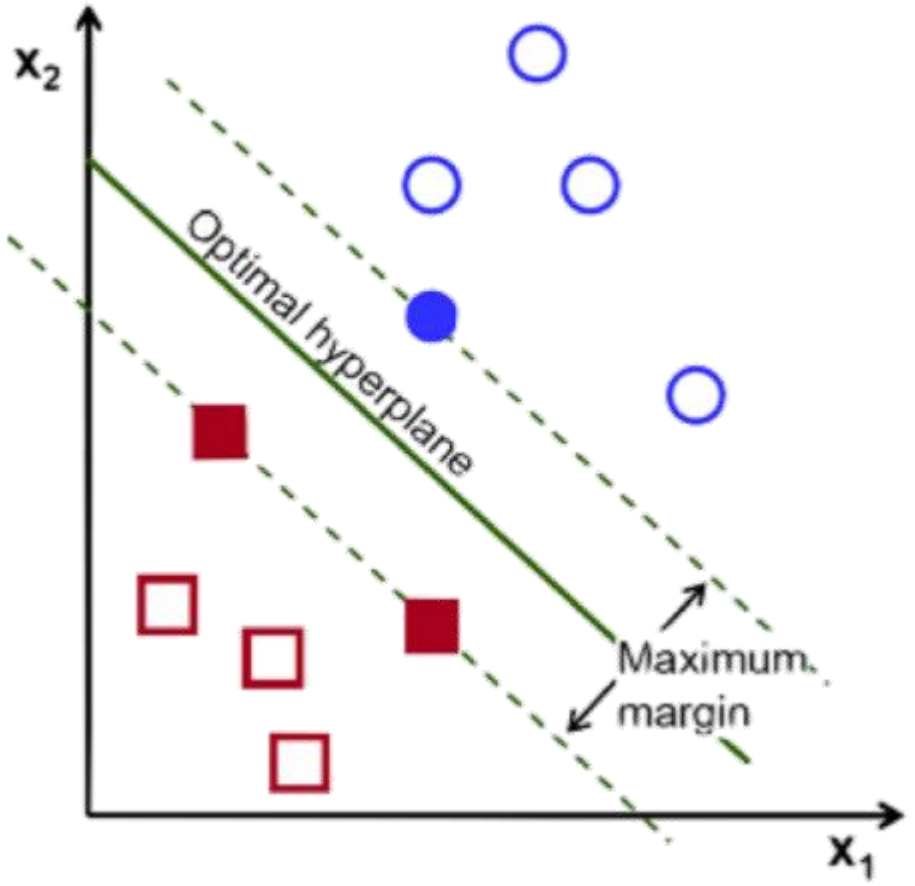
Restricciones en la frontera de decisión:

- Podemos considerar restricciones alternativas que prevengan el sobreajuste.
- Por ejemplo, podemos preferir una frontera de decisión que no 'favorezca' a ninguna clase (especialmente cuando las clases son aproximadamente igual de populosas).
- Geométricamente, esto significa elegir una frontera que maximice la distancia o margen entre la frontera y ambas clases.



Restricciones en la frontera de decisión:

- Maximizar la distancia de **todos los puntos** a la frontera de decisión, es exactamente lo mismo que maximizar la distancia a los **puntos más cercanos**.
- Los puntos más cercanos a la frontera de decisión se llaman **vectores de soporte**.
- Para cualquier plano, siempre podemos escalar la ecuación del hiperplano para que los vectores de soporte se encuentren en los planos.

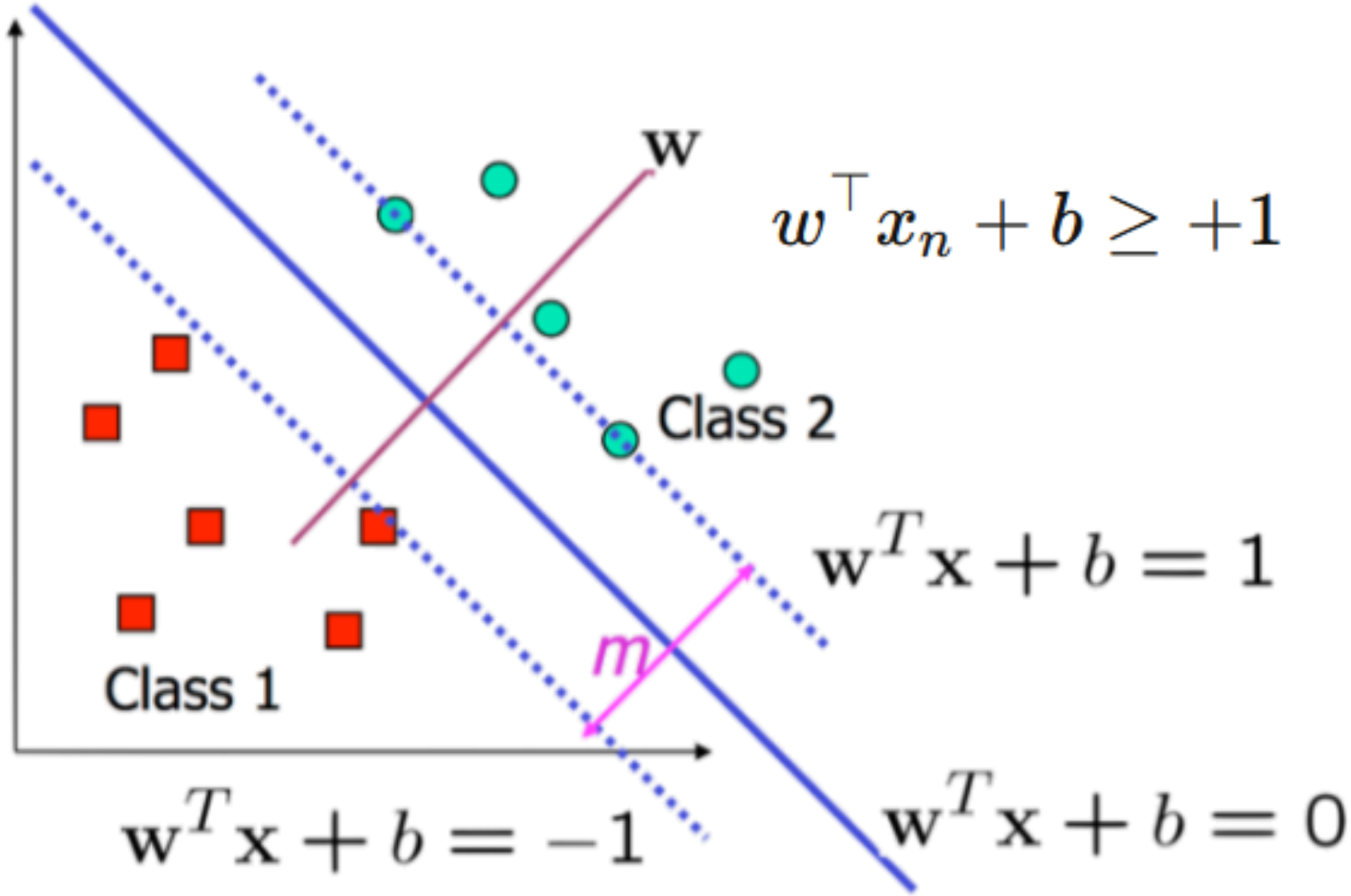


$$w^\top x_n + b \leq -1$$

Consideración general:

$$-1 \cdot (w^\top x_n + b) \geq -1 \cdot (-1)$$

$$y_n(w^\top x_n + b) \geq 1$$



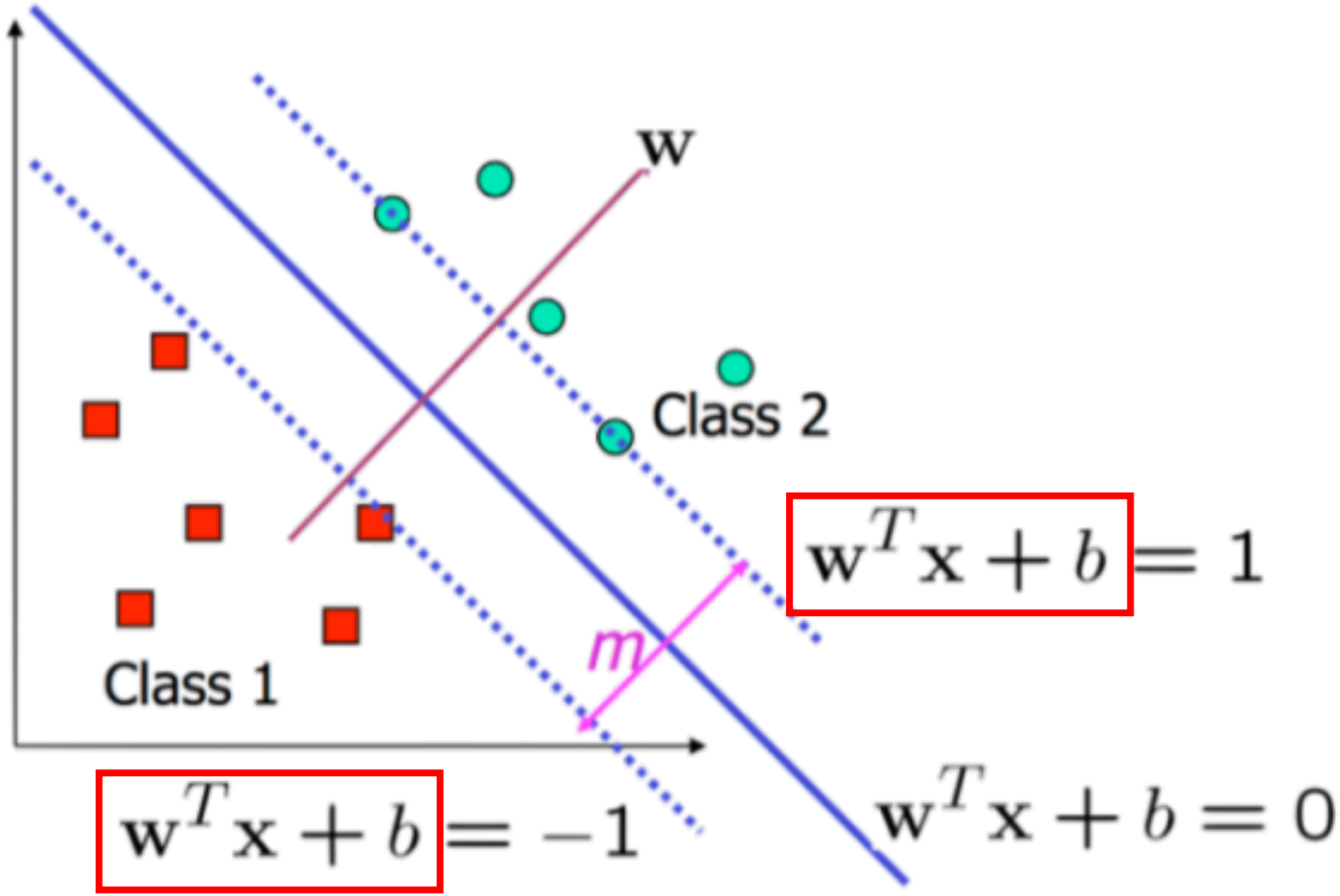
$$\mathbf{x}_1 = \mathbf{x}_0 - m \cdot \frac{\mathbf{w}}{\|\mathbf{w}\|}$$

$$\mathbf{w}^\top \mathbf{x}_1 + b = 0$$

$$\mathbf{w}^\top \left(\mathbf{x}_0 - m \cdot \frac{\mathbf{w}}{\|\mathbf{w}\|} \right) + b = 0$$

$$\mathbf{w}^\top \mathbf{x}_0 + b - m \cdot \frac{\|\mathbf{w}\|^2}{\|\mathbf{w}\|} = 0$$

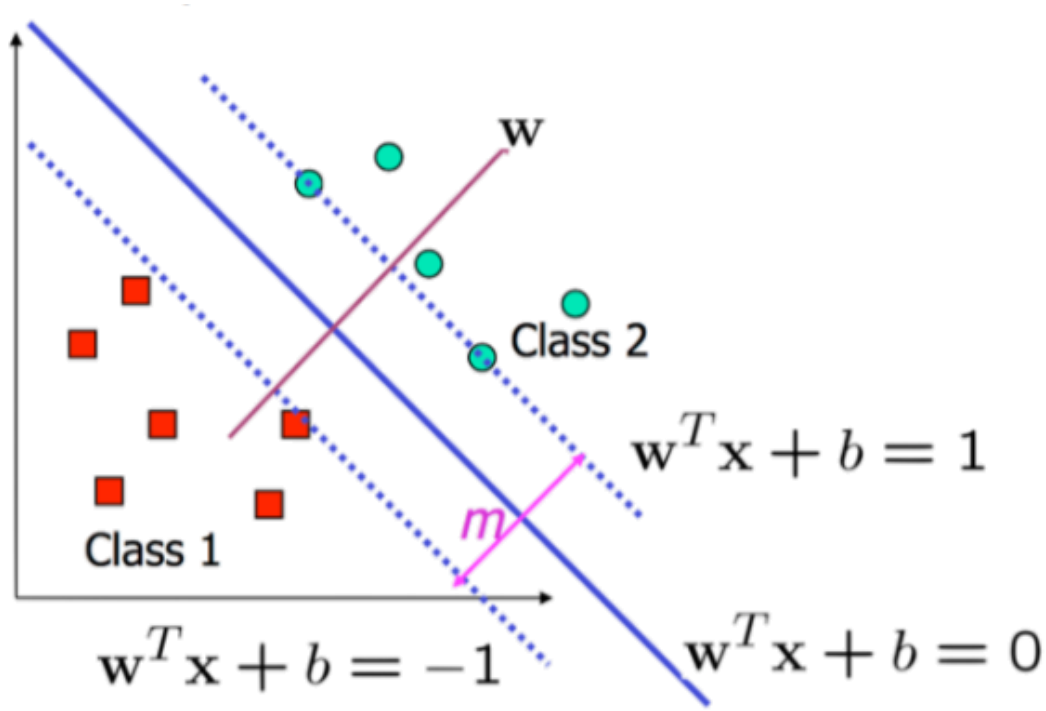
$$\mathbf{w}^\top \mathbf{x}_0 + b - m \cdot \|\mathbf{w}\| = 0$$



Clasificador de Vectores de Soporte

- Finalmente, podemos reformular nuestro problema de optimización:
  - encontrar una frontera de decisión que maximice la distancia a ambas clases
  - como la maximización del margen, m, mientras mantenemos cero clasificaciones erróneas:

$$\begin{cases} \max\limits_{w,b} \dfrac{2}{\|w\|} \\ \text{such that } y_n(w^\top x_n + b) \geq 1,\ n = 1, \ldots, N \end{cases}$$

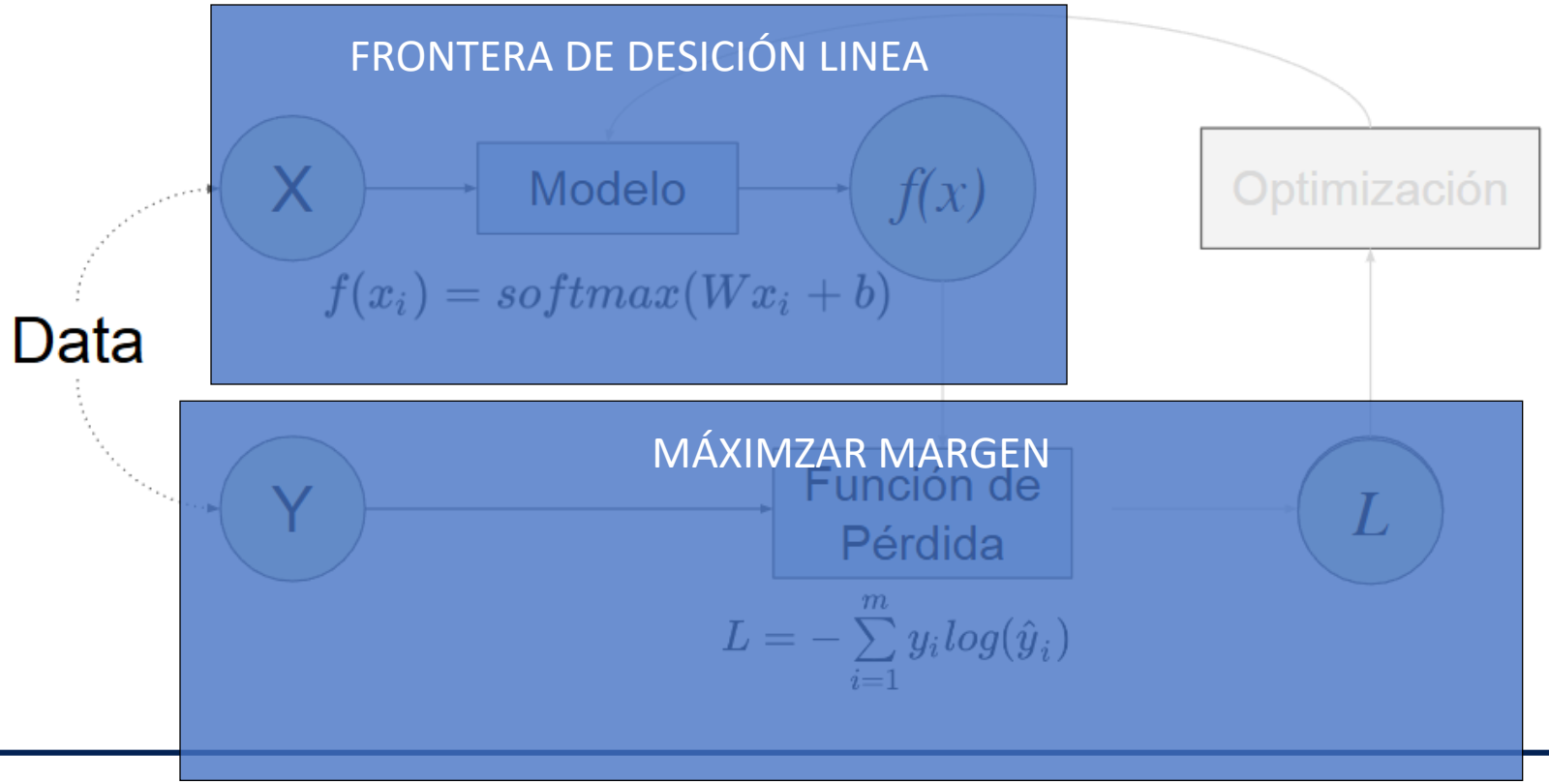


¿Cómo resolvemos este problema?

$$\begin{cases} \max\limits_{w,b} \dfrac{2}{\|w\|} \\ \text{such that } y_n(w^\top x_n + b) \geq 1, \ n = 1, \ldots, N \end{cases}$$

“Mejoramos” la formulación

~~Regresión logística~~ ~~Perceptrón~~ **SVM o Máquinas de Vectores de Soporte**





SVM y Optimización Convexa

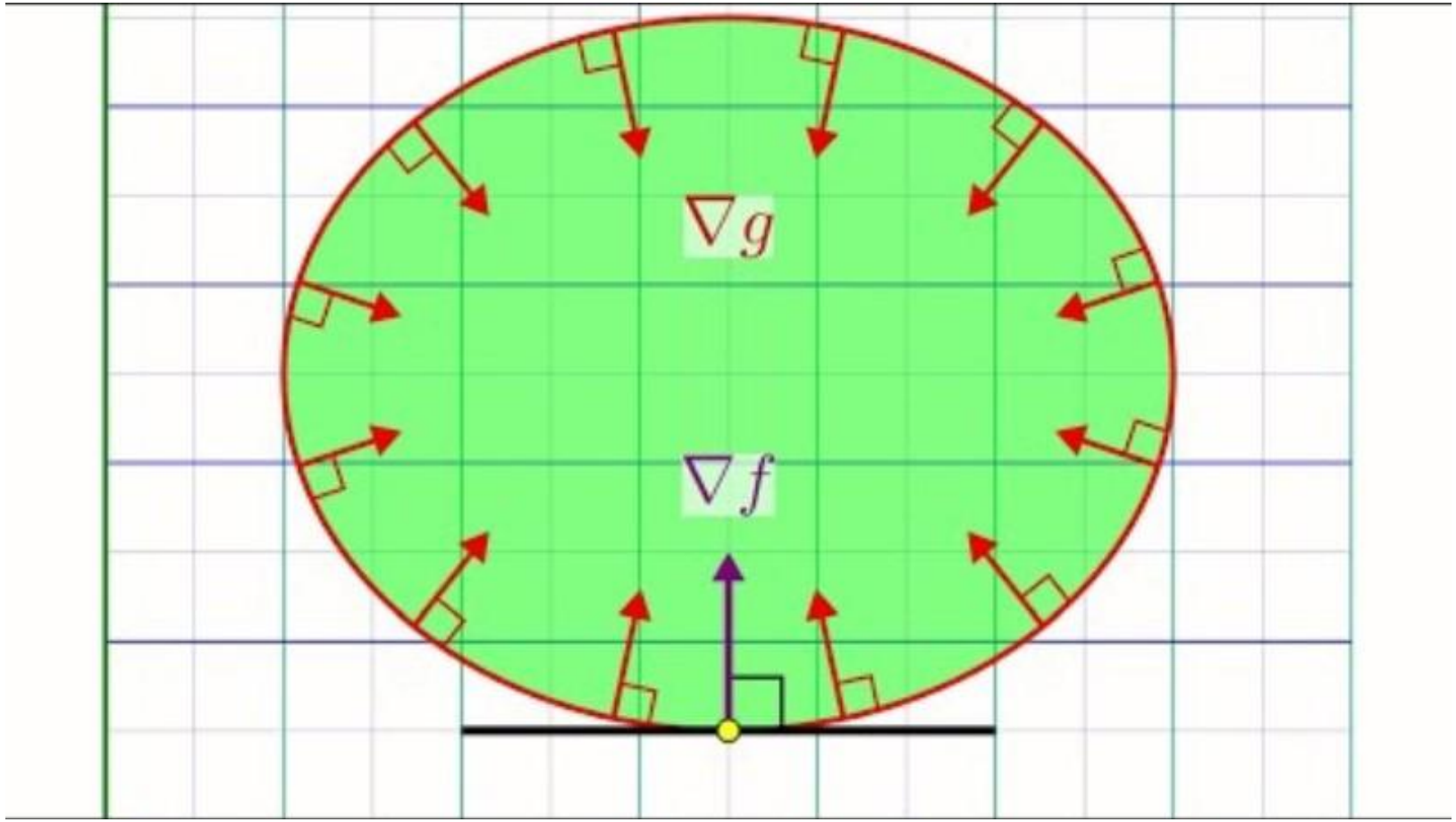
Como problema de optimización convexo, SVM ha sido extensamente estudiado y puede ser resuelto por una variedad de algoritmos:

- (Estocástico) libLinear
  - Convergencia rápida, costo computacional moderado
- (Codicioso) libSVM
  - Convergencia rápida, costo computacional moderado
- (Estocástico) Descenso de Gradiente Estocástico
  - Convergencia lenta, bajo costo computacional por iteración
- (Codicioso) Método Quasi-Newton
  - Convergencia muy rápida, alto costo computacional

¿Hay una forma matemática?

$$\begin{cases} \max\limits_{w,b} \dfrac{2}{\|w\|} \\ \text{such that } y_n(w^\top x_n + b) \geq 1, \ n = 1, \ldots, N \end{cases}$$

¡Multiplicadores de Lagrange!



¿Cómo utilizar Lagrange? (función objetivo)

$$\mathcal{L}(x, y, \lambda) = f(x, y) - \lambda \left(g(x, y) - c\right)$$

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial f}{\partial x} - \lambda \frac{\partial g}{\partial x} = 0 \quad \Rightarrow \quad \frac{\partial f}{\partial x} = \lambda \frac{\partial g}{\partial x}$$

$$\frac{\partial \mathcal{L}}{\partial y} = \frac{\partial f}{\partial y} - \lambda \frac{\partial g}{\partial y} = 0 \quad \Rightarrow \quad \frac{\partial f}{\partial y} = \lambda \frac{\partial g}{\partial y}$$

$$\frac{\partial \mathcal{L}}{\partial \lambda} = -\left(g(x, y) - c\right) = 0 \quad \Rightarrow \quad g(x, y) = c$$

Para SVM

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2}\|w\|^2 + \sum_{n=1}^{N} \alpha_n \left[1 - y_n(w^T x_n + b)\right]$$

$$\frac{\partial \mathcal{L}}{\partial w} = w - \sum_{n=1}^{N} \alpha_n y_n x_n = 0 \quad \Rightarrow \quad w = \sum_{n=1}^{N} \alpha_n y_n x_n$$

$$\frac{\partial \mathcal{L}}{\partial b} = -\sum_{n=1}^{N} \alpha_n y_n = 0 \quad \Rightarrow \quad \sum_{n=1}^{N} \alpha_n y_n = 0$$

$$\alpha_n \left[1 - y_n(w^T x_n + b)\right] = 0, \quad \alpha_n \geq 0$$

$$\mathcal{L}(w,b,\alpha) = \frac{1}{2}\|w\|^2 + \sum_{n=1}^{N} \alpha_n \left[1 - y_n(w^T x_n + b)\right]$$

$$\Rightarrow \quad w = \sum_{n=1}^{N} \alpha_n y_n x_n$$

$$\frac{1}{2}\|w\|^2 = \frac{1}{2} w^T w = \frac{1}{2}\left(\sum_{m=1}^{N} \alpha_m y_m x_m^T\right)\left(\sum_{k=1}^{N} \alpha_k y_k x_k\right) = \frac{1}{2}\sum_{m=1}^{N}\sum_{k=1}^{N} \alpha_m \alpha_k y_m y_k (x_m^T x_k)$$

$$\sum_{n=1}^{N} \alpha_n \left[1 - y_n(w^T x_n + b)\right] = \sum_{n=1}^{N} \alpha_n \left[1 - y_n w^T x_n - y_n b\right] = \sum_{n=1}^{N} \alpha_n \cdot 1 - \sum_{n=1}^{N} \alpha_n y_n w^T x_n - \sum_{n=1}^{N} \alpha_n y_n b$$

$$w^T \left(\sum_{n=1}^{N} \alpha_n y_n x_n\right)$$

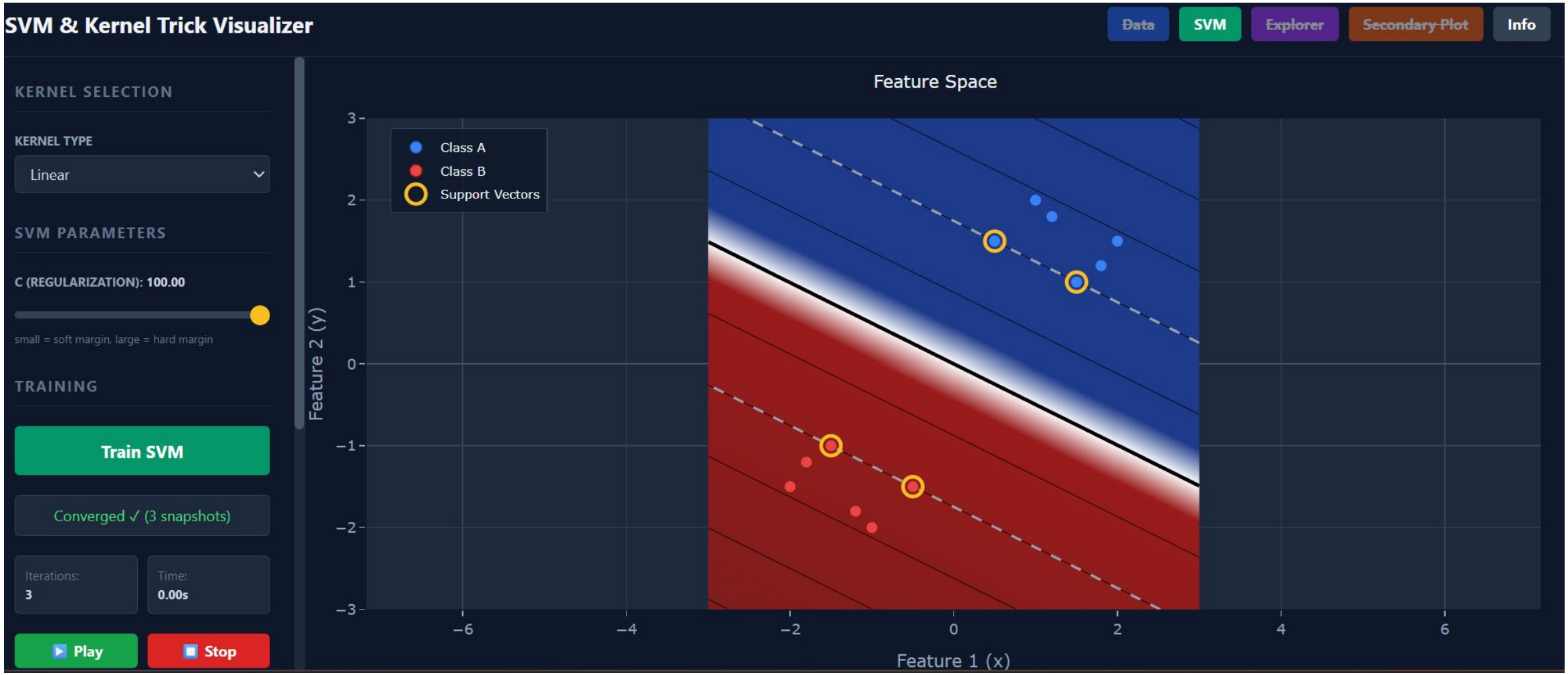
$$\mathcal{L}(w, b, \alpha) = \frac{1}{2}\|w\|^2 + \sum_{n=1}^{N} \alpha_n \left[1 - y_n(w^T x_n + b)\right]$$



$$\max_{\alpha} \sum_{n} \alpha_n - \frac{1}{2} \sum_{m} \sum_{k} \alpha_m \alpha_k y_m y_k \underbrace{\phi(x_m)^T \phi(x_k)}_{\text{inner product}}$$

$$f(x) = \text{sign}\left(\sum_{n} \alpha_n^* y_n \underbrace{\phi(x_n)^T \phi(x)}_{\text{inner product}} + b^* \right)$$

Demo Interactivo



(10 min) Break

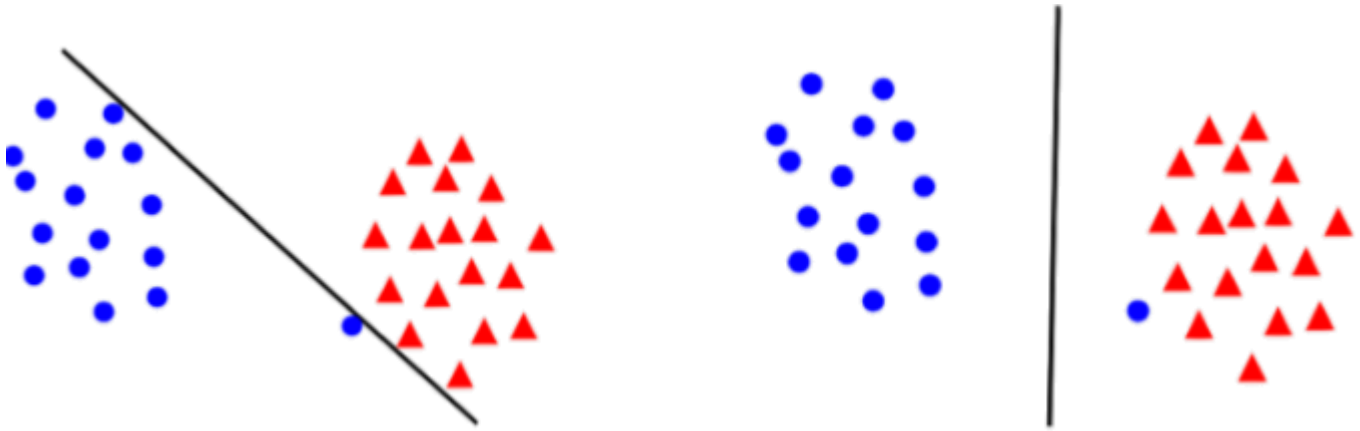




Clasificación de Datos Linealmente No Separables

Geometría de los Datos

- Maximizar el margen es una buena idea mientras asumamos que las clases subyacentes son linealmente separables y que los datos están libres de ruido.
- Si los datos son ruidosos, podríamos estar sacrificando generalización para minimizar el error de clasificación con un margen muy estrecho:



- Con cada frontera de decisión, hay un compromiso entre maximizar el margen y minimizar el error.

Clasificador de Vectores de Soporte

- Dado que queremos balancear maximizar el margen y minimizar el error, queremos usar una función objetivo que tome ambos en cuenta:

$$\begin{cases} \min\limits_{w,b} \|w\|^2 + \lambda \text{Error}(w,b) \\ \text{such that } y_n(w^\top x_n + b) \geq 1,\ n = 1, \ldots, N \end{cases}$$

- donde λ es un parámetro de intensidad.
- Entonces, ¿cómo deberíamos calcular el error para una frontera de decisión dada?

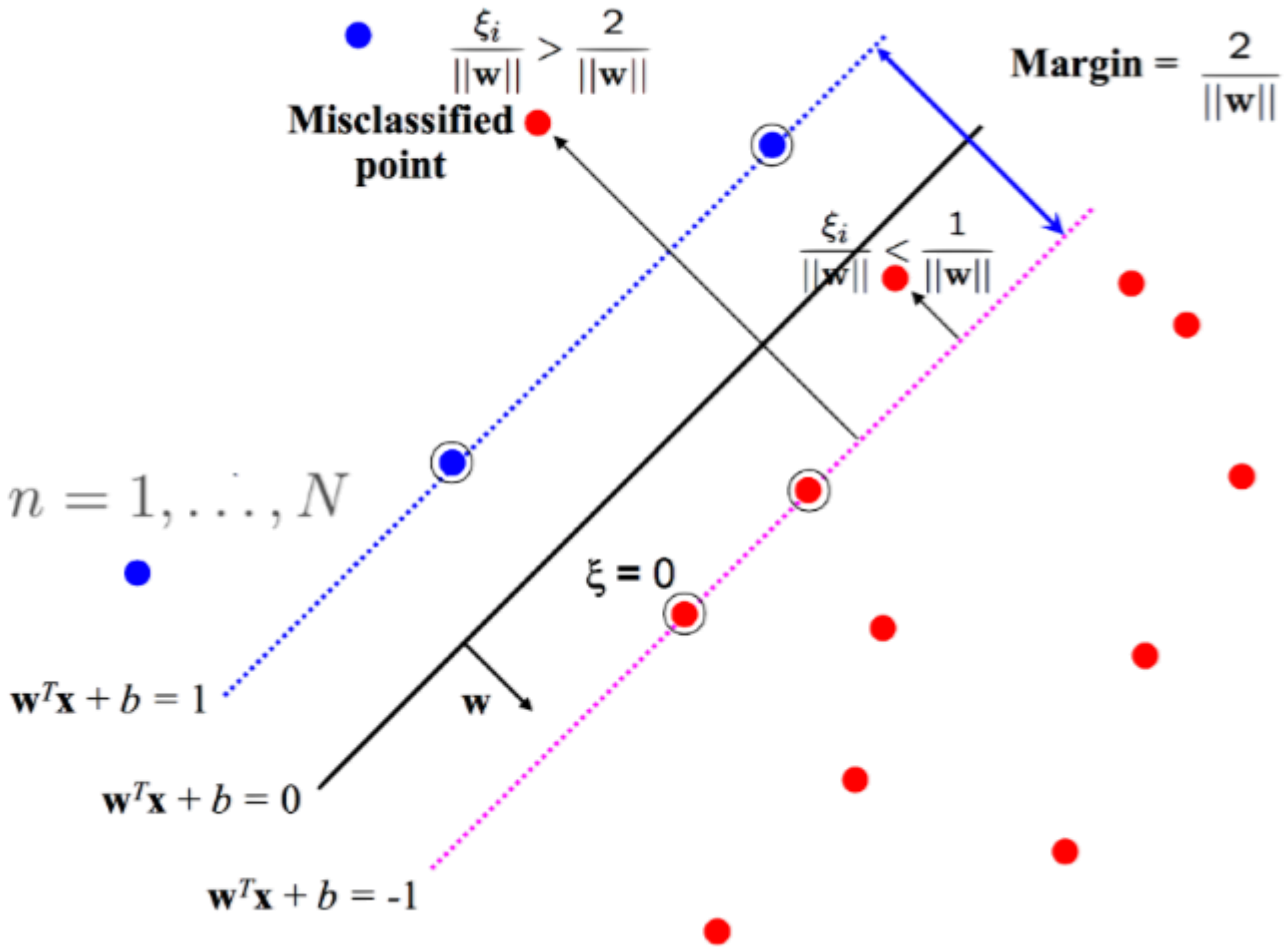
Clasificador de Vectores de Soporte

- Queremos expresar el error como una función de la distancia a la frontera de decisión.
- Recordemos que los vectores de soporte tienen distancia $1/||w||$ a la frontera de decisión.
- Queremos penalizar dos tipos de 'errores':
  - **(violación del margen)** puntos que están en el lado correcto de la frontera pero están dentro del margen. Tienen distancia $\xi/||w||$, donde $0 < \xi < 1$.
  - **(clasificación errónea)** puntos que están en el lado incorrecto de la frontera. Tienen distancia $\xi/||w||$, donde $\xi > 1$.
- Especificar una cantidad no negativa para ξ_n es equivalente a cuantificar el error en el punto x_n.

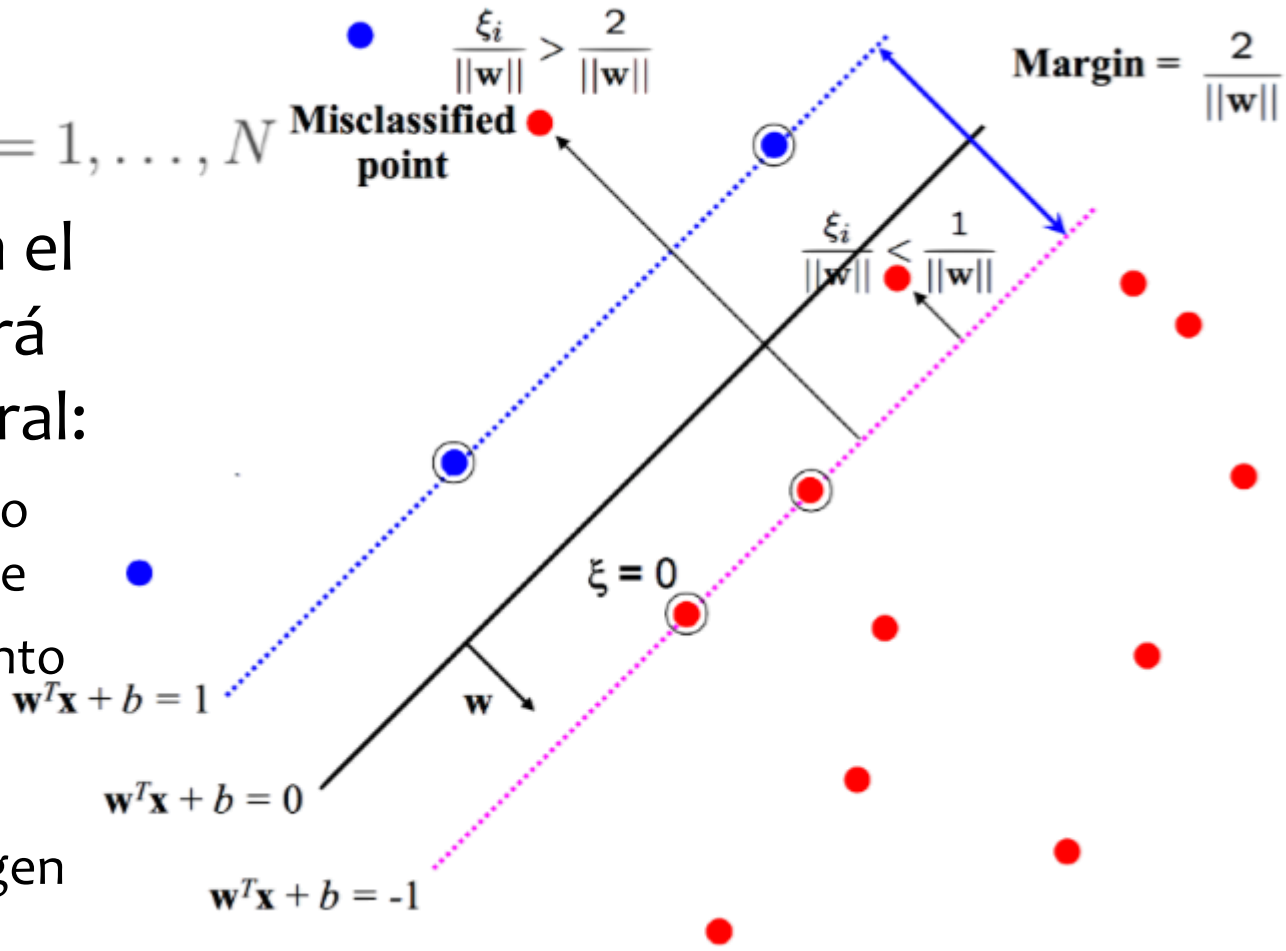
- Formalmente, incorporamos términos de error ξ_n en nuestro problema de optimización mediante:

$$\begin{cases} \min\limits_{\xi_n \in \mathbb{R}^+, w, b} \|w\|^2 + \lambda \sum\limits_{n=1}^{N} \xi_n \\ \text{such that } y_n(w^\top x_n + b) \geq 1 - \xi_n, \ n = 1, \ldots, N \end{cases}$$

- La solución a este problema se llama clasificación de vectores de soporte de margen suave o simplemente clasificación de vectores de soporte.



$$\begin{cases} \min\limits_{\xi_n \in \mathbb{R}^+, w, b} \|w\|^2 + \lambda \sum\limits_{n=1}^{N} \xi_n \\ \text{such that } y_n(w^\top x_n + b) \geq 1 - \xi_n, \; n = 1, \ldots, N \end{cases}$$



- Elegir diferentes valores para λ en el problema de optimización nos dará diferentes clasificadores. En general:
- λ pequeña penaliza menos los errores y por lo tanto el clasificador tendrá un margen grande
- λ grande penaliza más los errores y por lo tanto el clasificador aceptará márgenes estrechos para mejorar la clasificación
- establecer λ = ∞ produce la solución de margen duro

Extensión a Fronteras No Lineales

¿Qué utilizamos para extender un modelo lineal a uno no lineal?

Regresión Polinomial

- Dado un conjunto de entrenamiento: $\{(x_1, y_1),\ldots,(x_n, y_n)\}$
- con un predictor de valor real único, podemos ver el ajuste de un modelo polinomial de 2º grado: $w_0 + w_1x + w_2x^2$
- en los datos como el proceso de encontrar la mejor curva cuadrática que se ajuste a los datos. Pero en la práctica, primero expandimos la dimensión de características del conjunto de entrenamiento:

$$x_n \mapsto (x_n^0, x_n^1, x_n^2)$$

- y entrenamos un **modelo lineal** en los datos expandidos:

$$\{(x_n^0, x_n^1, x_n^2, y_1),\ldots,(x_n^0, x_n^1, x_n^2, y_n)\}$$

Transformación de los Datos

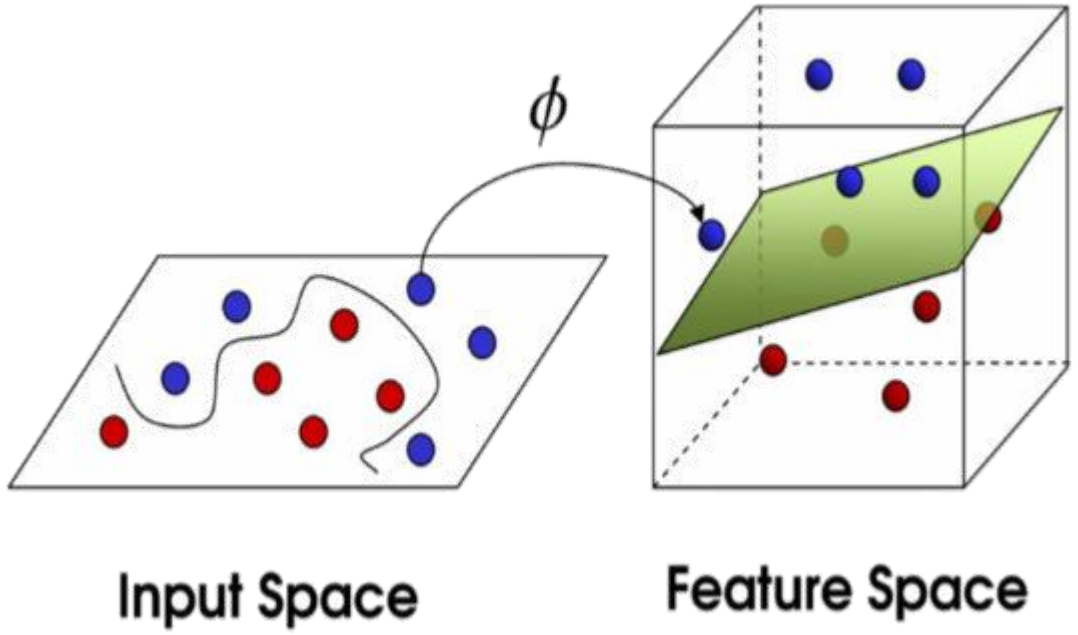
- La observación clave es que entrenar un modelo polinomial es solo entrenar un modelo lineal en datos con predictores transformados.
- En nuestro ejemplo anterior, transformar los datos para ajustar un modelo polinomial de 2º grado requiere un mapeo:

$$\phi : \mathbb{R} \rightarrow \mathbb{R}^3$$
$$\phi(x) = (x^0, x^1, x^2)$$

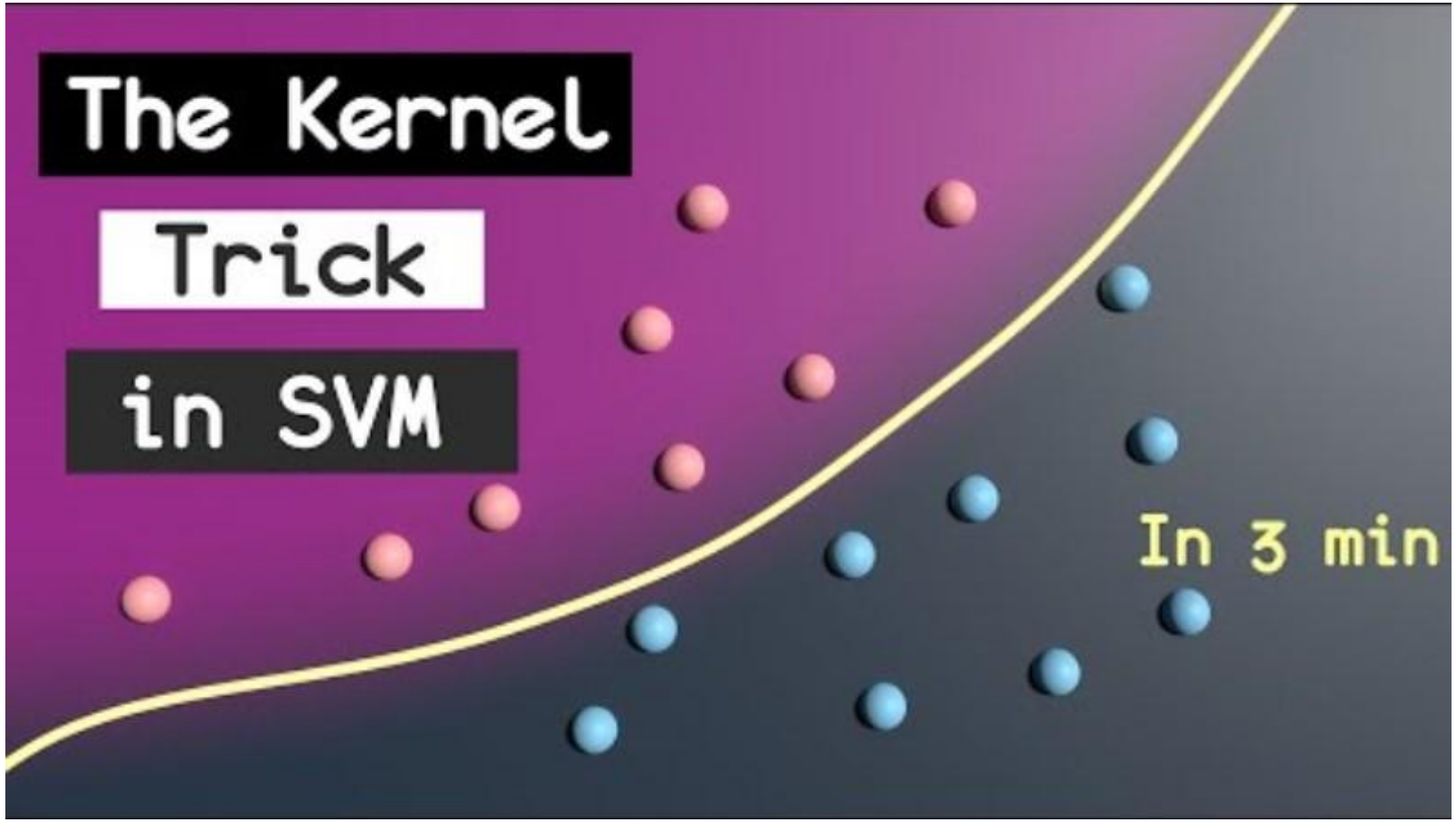
- donde $\mathbb{R}$ se llama el **espacio de entrada**, $\mathbb{R}^3$ se llama el **espacio de características**.
- Mientras que la respuesta puede no tener una correlación lineal en el espacio de entrada $\mathbb{R}$, puede tenerla en el espacio de características $\mathbb{R}^3$.

SVM con Fronteras de Decisión No Lineales

La misma intuición se aplica a la clasificación: mientras que la respuesta puede no ser linealmente separable en el espacio de entrada, puede serlo en un espacio de características después de una transformación sofisticada:



Visualización





SVM con Fronteras de Decisión No Lineales

En lugar de modificar la definición de SVM para acomodar fronteras de decisión no lineales, mapeamos los datos a un espacio de características en el cual las clases son linealmente separables (o casi separables):

- Aplicar transformación $\phi: \mathbb{R}^J \rightarrow \mathbb{R}^{J'}$ en datos de entrenamiento

  $$x_n \rightarrow \phi(x_n)$$

  donde típicamente J' es mucho mayor que J.
- Entrenar un SVC en los datos transformados
  $\{(\phi(x_1), y_1), (\phi(x_2), y_2), \ldots, (\phi(x_n), y_n)\}$

Producto Interno

- Dado que el espacio de características $\mathbb{R}^{j'}$ es potencialmente extremadamente de alta dimensión, calcular ϕ explícitamente puede ser costoso.
- En su lugar, notamos que calcular ϕ es innecesario. Recordemos que entrenar un SVM involucra resolver el problema de optimización.

$$\max_{\alpha_n \geq 0,\ \sum_n \alpha_n y_n = 0} \sum_n \alpha_n - \frac{1}{2} \sum_{n,m=1}^{N} y_n y_m \alpha_n \alpha_m x_n^\top x_m \qquad \text{(Forma Dual)}$$

- En lo anterior, **solo estamos interesados en calcular productos internos $\phi(x_n)^T\phi(x_m)$ en el espacio de características** y no las cantidades $\phi(x_n)$ en sí mismas.

El Truco del Kernel o Núcleo

- El producto interno entre dos vectores es una medida de la similitud de los dos vectores.

Definition

Given a transformation $\phi : \mathbb{R}^J \to \mathbb{R}^{J'}$, from input space $\mathbb{R}^J$ to feature space $\mathbb{R}^{J'}$, the function $K : \mathbb{R}^J \times \mathbb{R}^J \to \mathbb{R}$ defined by

$$K(x_n, x_m) = \phi(x_n)^\top \phi(x_m), \quad x_n, x_m \in \mathbb{R}^J$$

is called the ***kernel function*** of ϕ.

Generally, ***kernel function*** may refer to any function $K : \mathbb{R}^J \times \mathbb{R}^J \to \mathbb{R}$ that measure the similarity of vectors in $\mathbb{R}^J$, without explicitly defining a transform ϕ.

Kernels o Núcleos: Motivación

La mayoría de los problemas del mundo real presentan datos que no son linealmente separables.

Ejemplo: representación de píxeles para el reconocimiento facial:



Q: Cuando sus datos **no son linealmente separables**, ¿cómo puede seguir utilizando un clasificador lineal?

A: Preprocesar los datos para producir **características no lineales**



Núcleos: Motivación

- Motivación nº 1: Funciones ineficaces
  - Los datos no linealmente separables requieren una representación **de alta dimensión**
  - Podría ser **prohibitivamente** caro de computar o almacenar
- Motivación nº 2: Métodos basados en la memoria
  - k--Nearest Neighbors (KNN) para el reconocimiento facial permite una **métrica de distancia** entre imágenes –sin necesidad de preocuparse en absoluto por la restricción de linealidad

Métodos de núcleo

- **Idea clave:**
  1. Reescriba el algoritmo de forma que sólo trabajemos con **productos escalares** $x^T z$ de vectores de características
  2. Sustituya los **productos punto** $x^T z$ por una **función núcleo** $k(x, z)$

- El núcleo $k(x,z)$ puede ser **cualquier** definición legal de un producto punto: $k(x, z) = \varphi(x)^T \varphi(z)$ para cualquier función $\varphi: \mathcal{X} \rightarrow \mathbf{R}^D$

  Así que sólo calculamos el producto punto φ **implícitamente**

- Este **"truco del núcleo"** puede aplicarse a muchos algoritmos:
  - clasificación: perceptron, SVM, ...
  - regresión: regresión ridge, ...
  - agrupación: k--means, ...

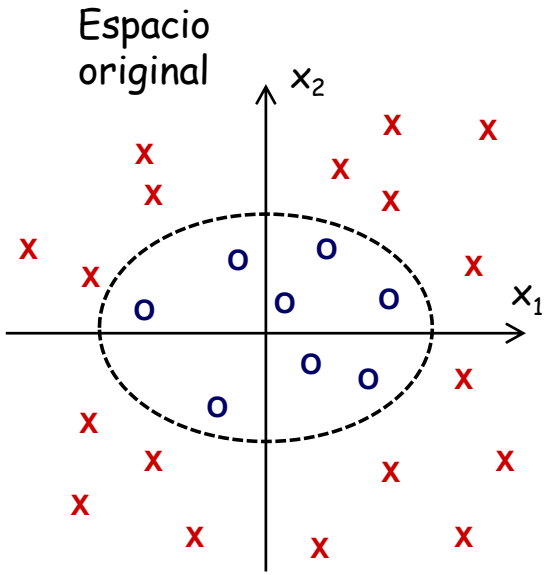
Ejemplo: Núcleo polinómico

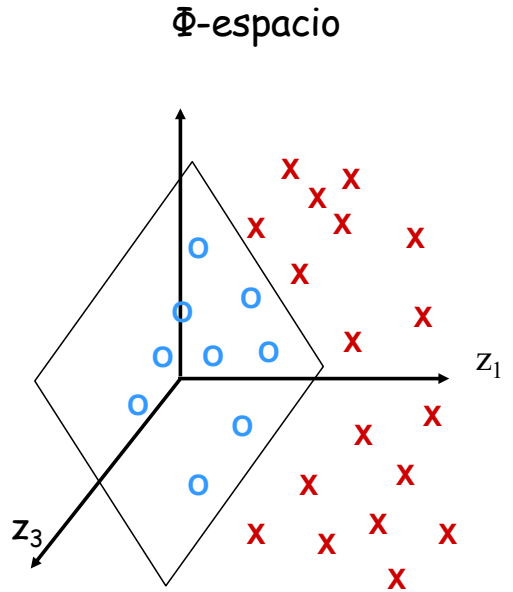
Para n=2, d=2, el núcleo $K(x, z) = (x \cdot z)^d$ corresponde a

$$\phi: R^2 \to R^3, (x_1, x_2) \to \Phi(x) = (x_1^2, x_2^2, \sqrt{2}x_1x_2)$$

$$\phi(x) \cdot \phi(z) = \left(x_1^2, x_2^2, \sqrt{2}x_1x_2\right) \cdot (z_1^2, z_2^2, \sqrt{2}z_1z_2)$$

$$= (x_1z_1 + x_2z_2)^2 = (x \cdot z)^2 = K(x, z)$$





Ejemplo: Núcleo polinómico

El espacio de características puede crecer mucho y muy rápido....

¡¡¡¡Es crucial pensar en ϕ implícito, no explícito!!!!

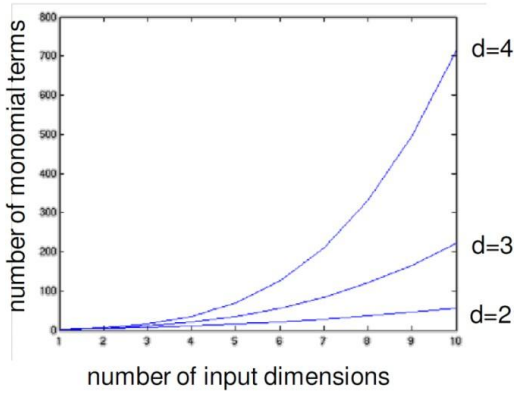
Núcleo polinómico de grado d, $k(x,z) = (x^\top z)^d = \phi(x) \cdot \phi(z)$

$x_1^d,\ x_1 x_2 \dots x_d,\ x_1^2 x_2 \dots x_{d-1}$

- El número total de esta característica es

$$\binom{d+n-1}{d} = \frac{(d+n-1)!}{d!\,(n-1)!}$$

- $d = 6, n = 100$, hay 1600 millones de términos



¡$O(n)$ *computation*!

$$k(x,z) = x(z^{\top d}) = \phi(x) \cdot \phi(z)$$

Ejemplos de núcleos

Nota al margen: ¡El espacio de características puede no ser único!

Representación explícita n° 1:

$$\phi: R^2 \to R^3, (x_1, x_2) \to \Phi(x) = (x_1^2, x_2^2, \sqrt{2}x_1x_2)$$

$$\phi(x) \cdot \phi(z) = \left(x_1^2, x_2^2, \sqrt{2}x_1x_2\right) \cdot (z_1^2, z_2^2, \sqrt{2}z_1z_2)$$

$$= (x_1z_1 + x_2z_2)^2 = (x \cdot z)^2 = K(x, z)$$

Representación explícita n° 2:

$$\phi: R^2 \to R^4, (x_1, x_2) \to \Phi(x) = (x_1^2, x_2^2, x_1x_2, x_2x_1)$$

$$\phi(x) \cdot \phi(z) = (x_1^2, x_2^2, x_1x_2, x_2x_1) \cdot (z_1^2, z_2^2, z_1z_2, z_2z_1)$$

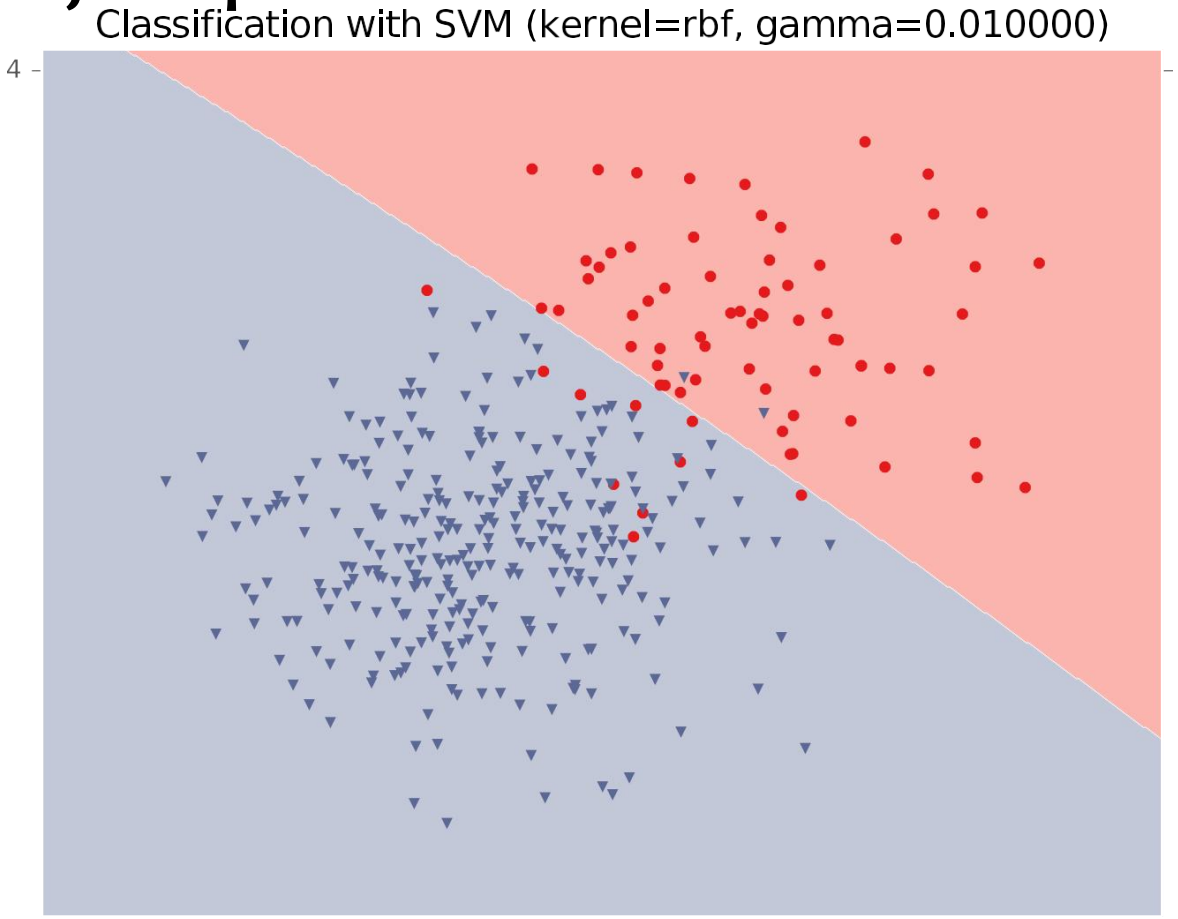
$$= (x \cdot z)^2 = K(x, z)$$

¡Estas dos representaciones de caracteristicas diferentes corresponden a la misma función del núcleo!

Ejemplos de núcleos

Nombre	Función núcleo (producto punto implícito)	Espacio de características (producto punto explícito)
Lineal	$K(\mathbf{x}, \mathbf{z}) = \mathbf{x}^T \mathbf{z}$	Igual que el espacio de entrada original
Polinomio (v1)	$K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^T \mathbf{z})^d$	Todos los polinomios **de** grado d
Polinomio (v2)	$K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^T \mathbf{z} + 1)^d$	Todos los polinomios **hasta** grado d
Gaussiano	$K(\mathbf{x}, \mathbf{z}) = \exp(-\frac{\|\|\mathbf{x} - \mathbf{z}\|\|_2^2}{2\sigma^2})$	Espacio de dimensión infinita
Núcleo tangente hiperbólico (sigmoide)	$K(\mathbf{x}, \mathbf{z}) = \tanh(\alpha \mathbf{x}^T \mathbf{z} + c)$	(Con SVM, esto equivale a una red neuronal de 2 capas)

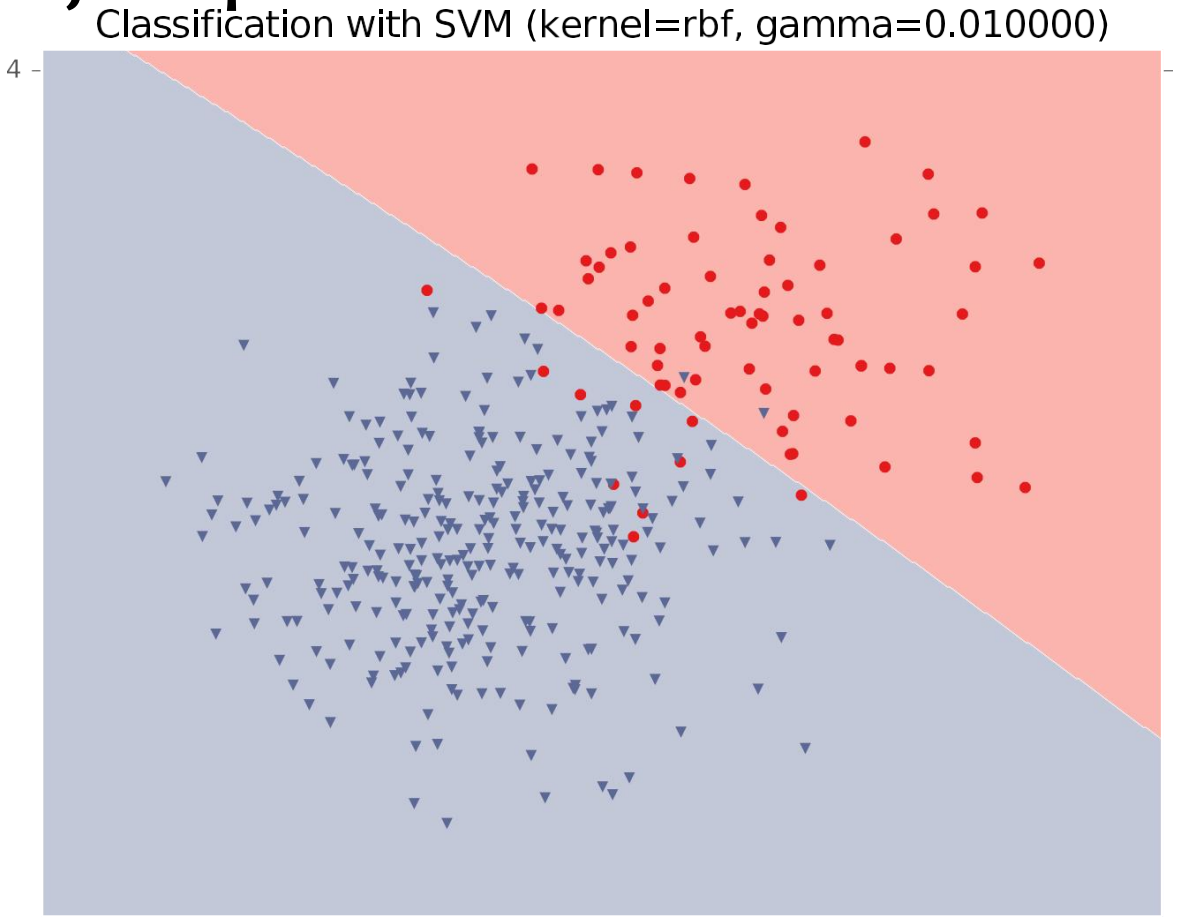
Ejemplo de núcleo RBF



Núcleo RBF:

$$K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \exp(-\gamma||\mathbf{x}^{(i)} - \mathbf{x}^{(j)}||_2^2)$$

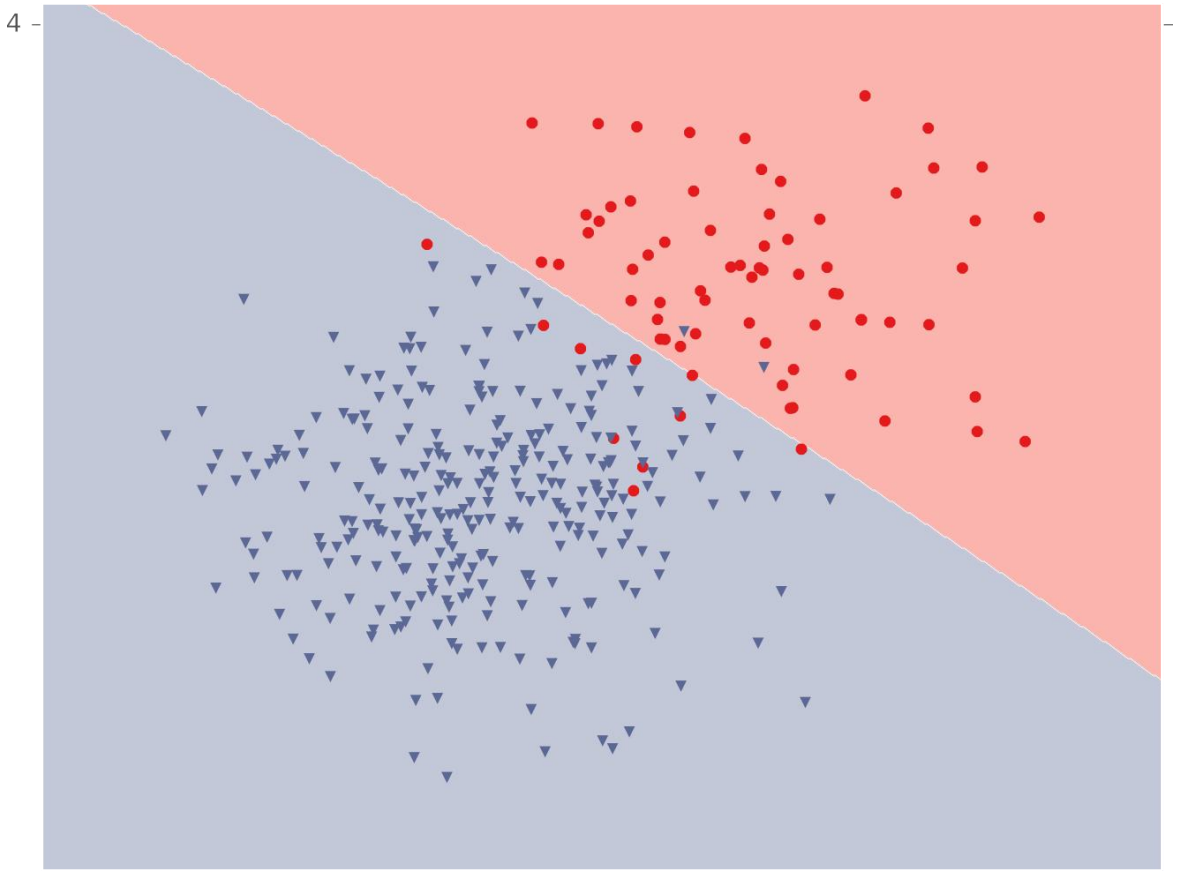
Ejemplo de núcleo RBF



Núcleo RBF:

$$K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \exp(-\gamma||\mathbf{x}^{(i)} - \mathbf{x}^{(j)}||_2^2)$$

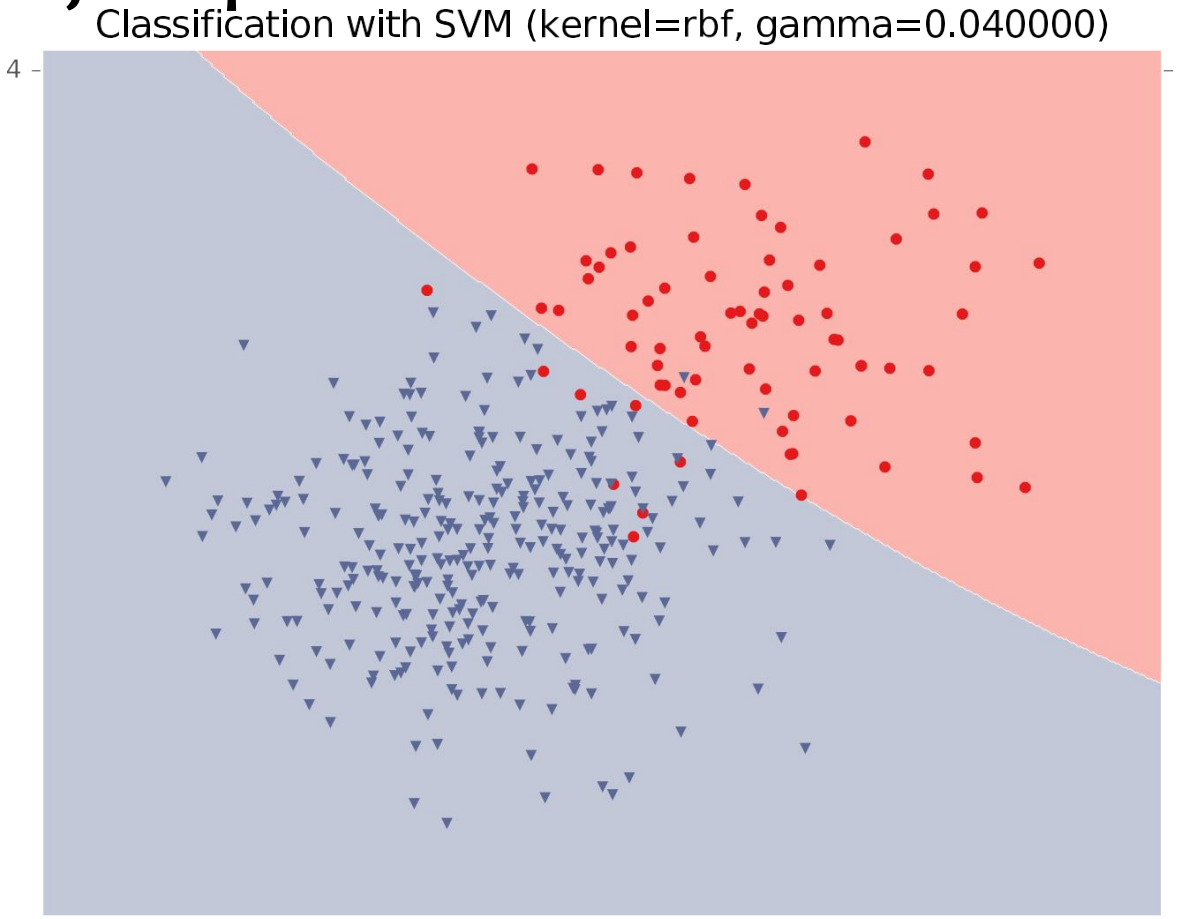
Ejemplo de núcleo RBF



Núcleo RBF:

$$K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \exp(-\gamma||\mathbf{x}^{(i)} - \mathbf{x}^{(j)}||_2^2)$$

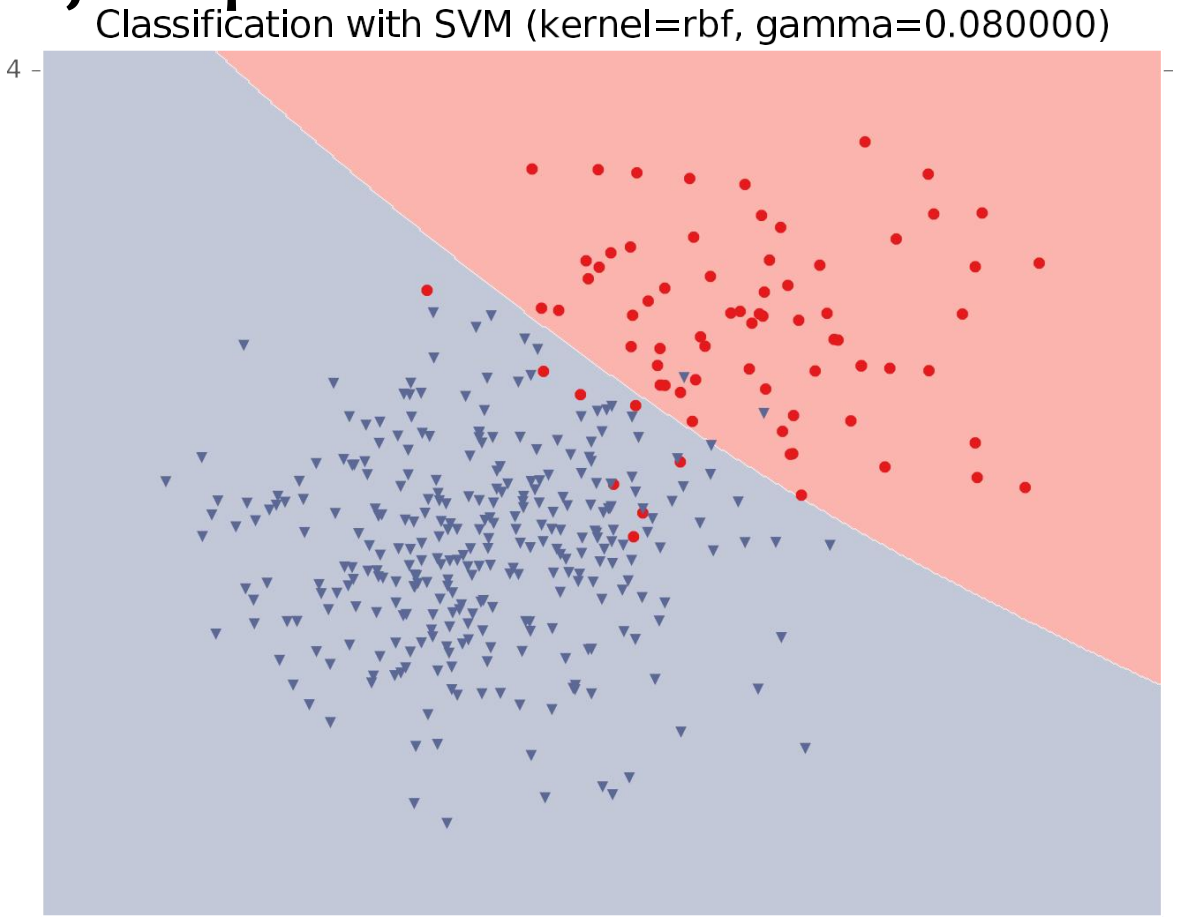
Ejemplo de núcleo RBF



Núcleo RBF:

$$K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \exp(-\gamma||\mathbf{x}^{(i)} - \mathbf{x}^{(j)}||_2^2)$$

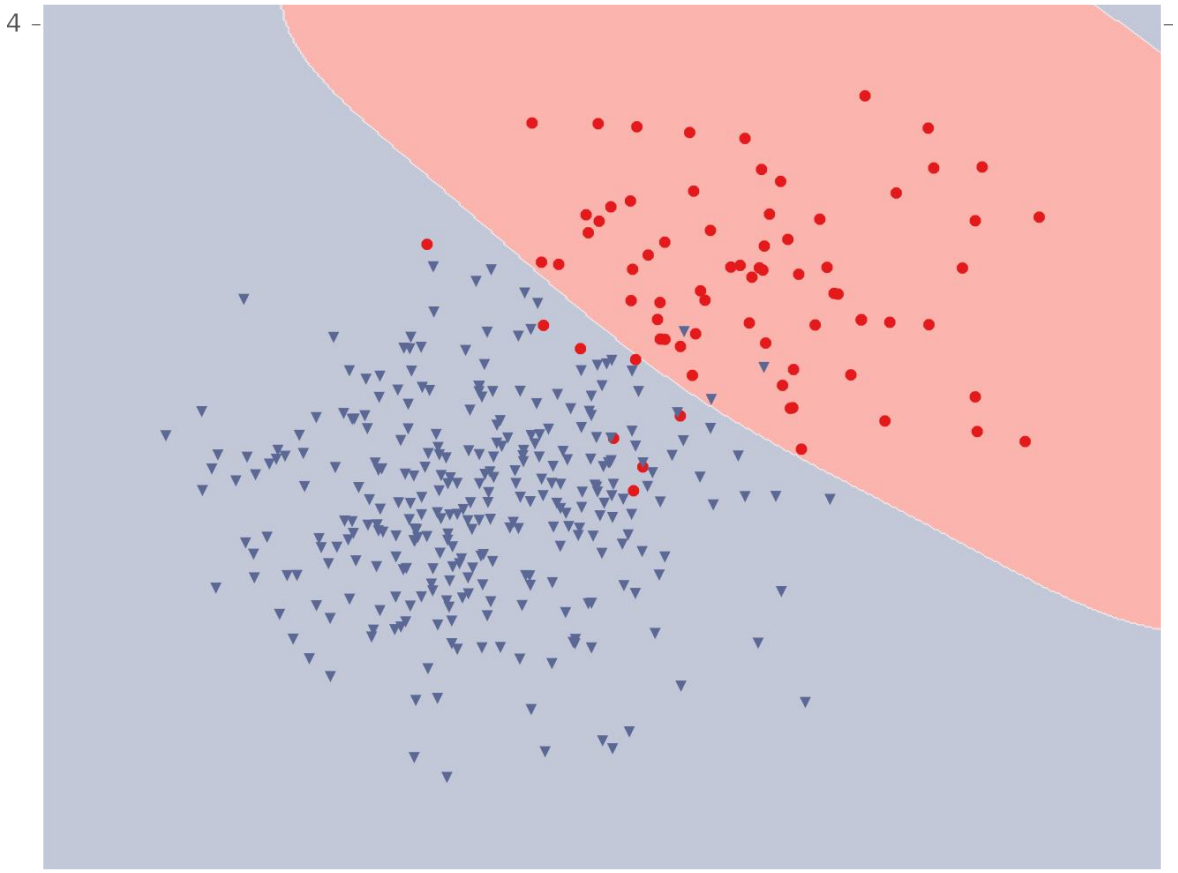
Ejemplo de núcleo RBF



Núcleo RBF:

$$K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \exp(-\gamma||\mathbf{x}^{(i)} - \mathbf{x}^{(j)}||_2^2)$$

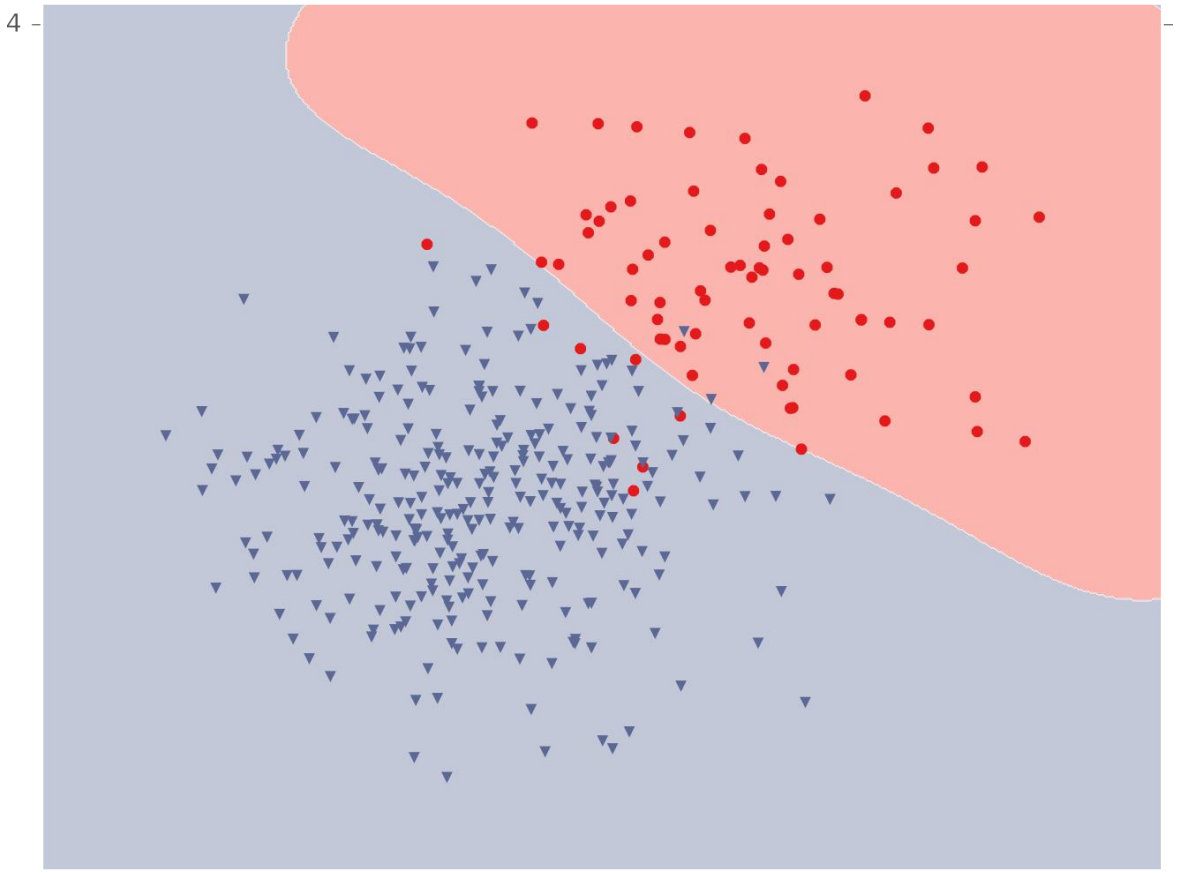
Ejemplo de núcleo RBF



Núcleo RBF:

$$K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \exp(-\gamma||\mathbf{x}^{(i)} - \mathbf{x}^{(j)}||_2^2)$$

Ejemplo de núcleo RBF



Núcleo RBF:

$$K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \exp(-\gamma ||\mathbf{x}^{(i)} - \mathbf{x}^{(j)}||_2^2)$$

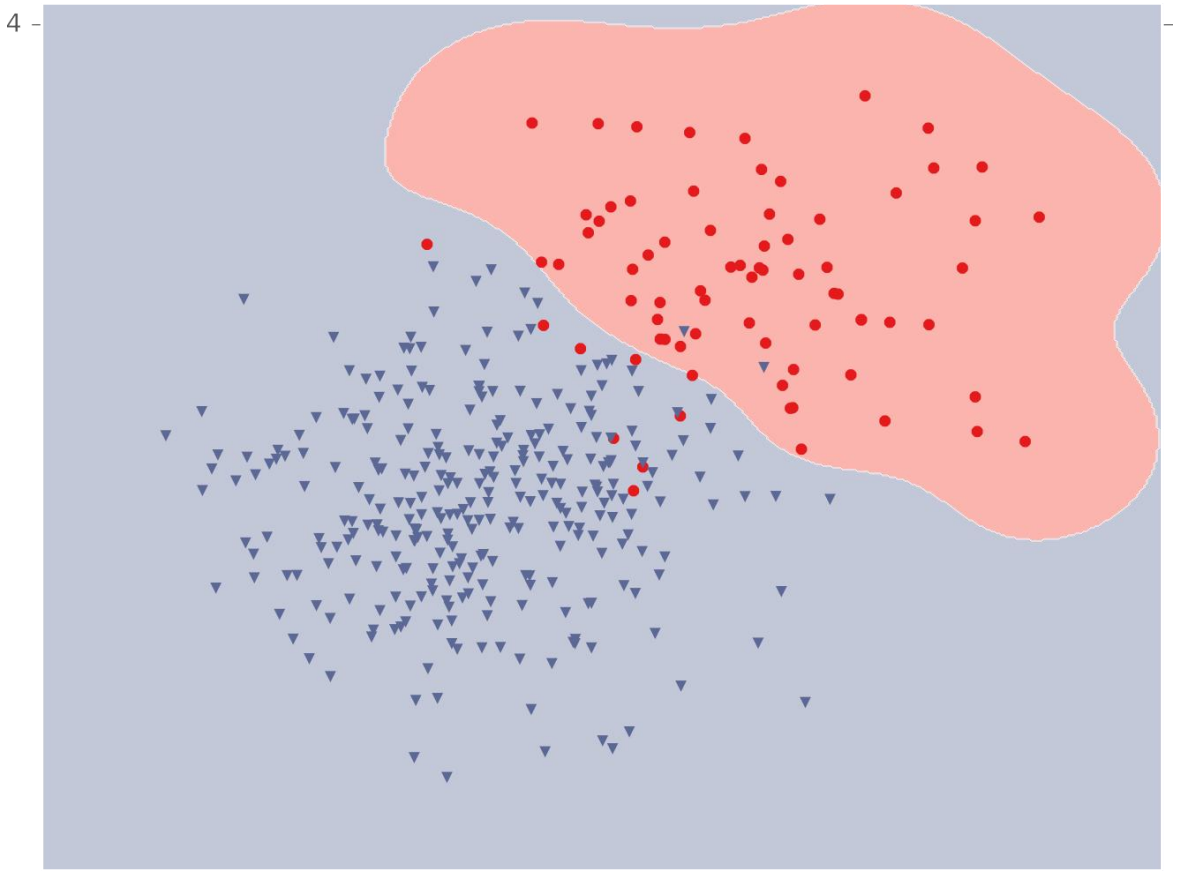
Ejemplo de núcleo RBF



Núcleo RBF:

$$K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \exp(-\gamma||\mathbf{x}^{(i)} - \mathbf{x}^{(j)}||_2^2)$$

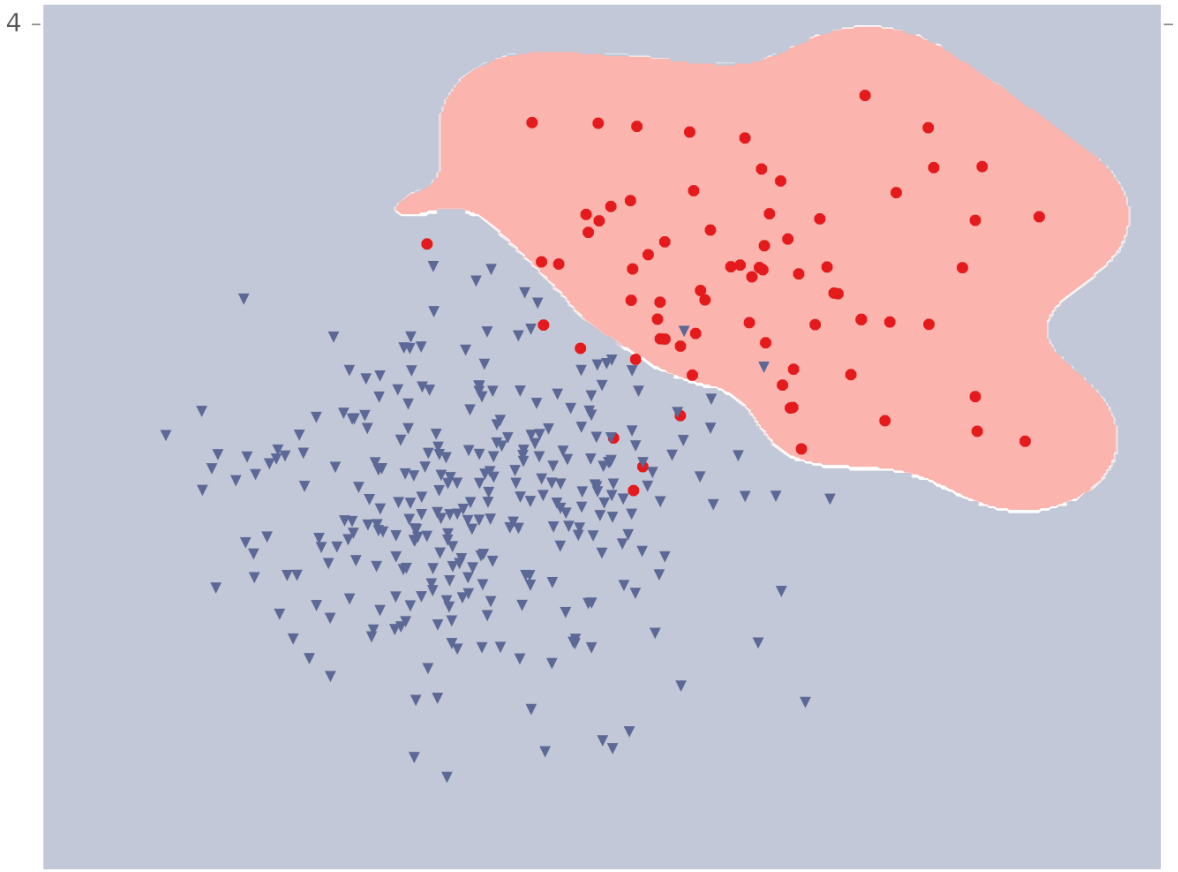
Ejemplo de núcleo RBF



Núcleo RBF:

$$K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \exp(-\gamma||\mathbf{x}^{(i)} - \mathbf{x}^{(j)}||_2^2)$$

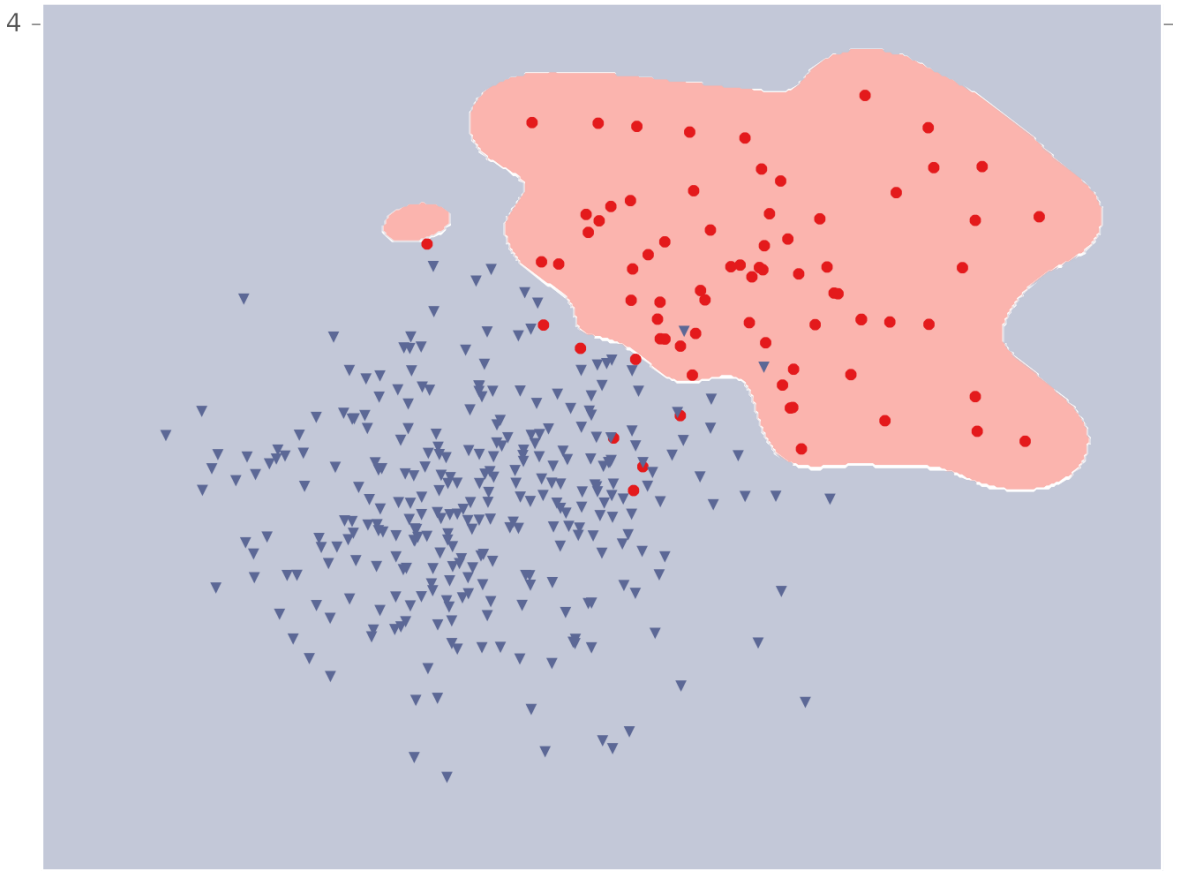
Ejemplo de núcleo RBF



Núcleo RBF:

$$K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \exp(-\gamma||\mathbf{x}^{(i)} - \mathbf{x}^{(j)}||_2^2)$$

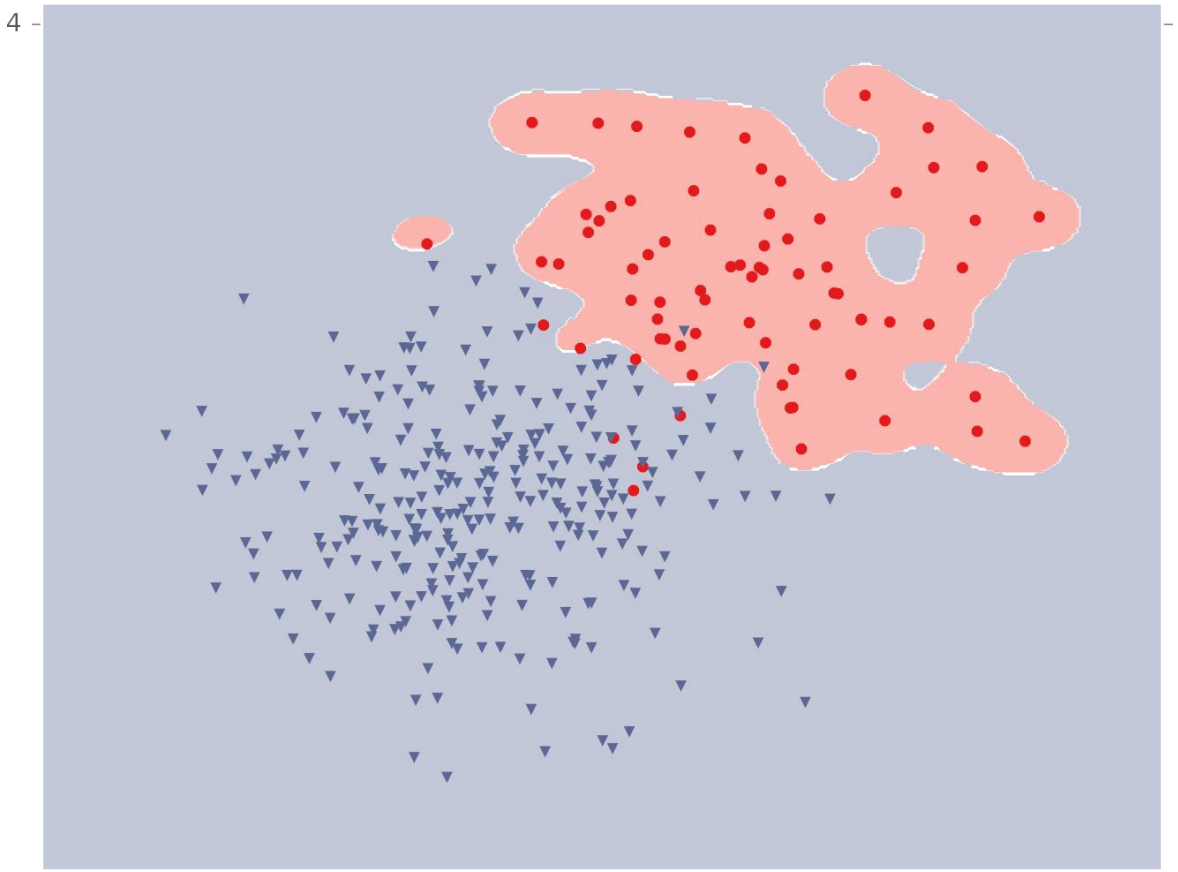
Ejemplo de núcleo RBF



Núcleo RBF:

$$K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \exp(-\gamma||\mathbf{x}^{(i)} - \mathbf{x}^{(j)}||_2^2)$$

Ejemplo de núcleo RBF

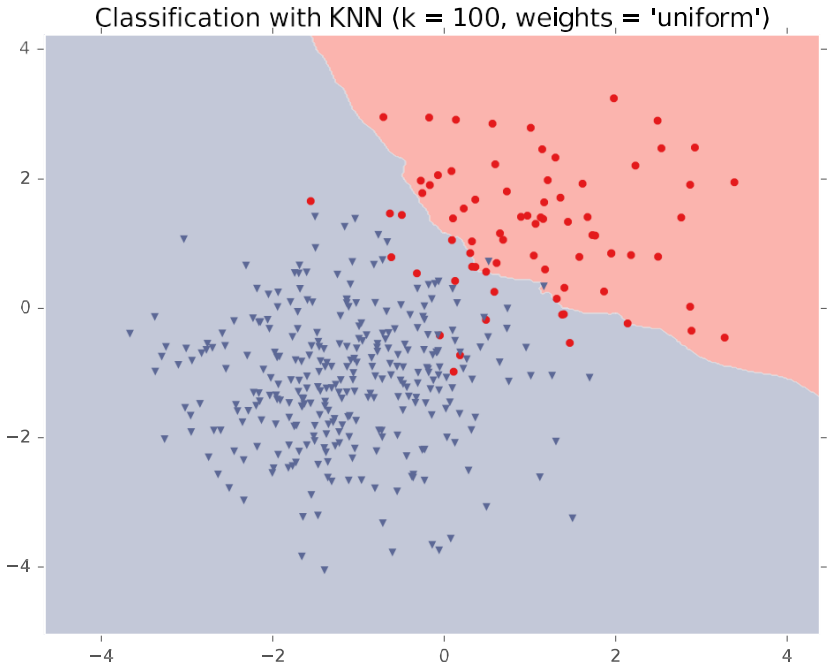


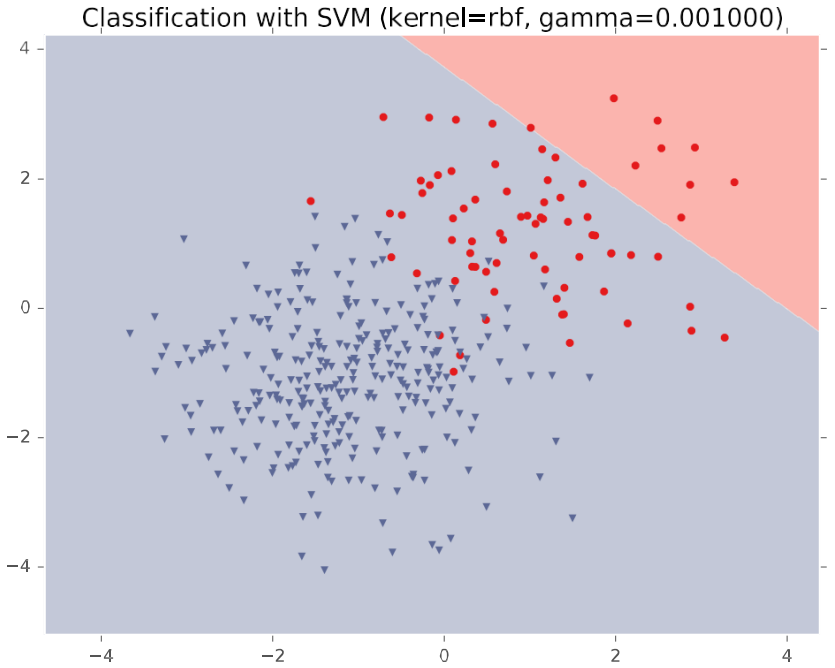
Núcleo RBF:

$$K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \exp(-\gamma||\mathbf{x}^{(i)} - \mathbf{x}^{(j)}||_2^2)$$

Ejemplo de núcleo RBF

KNN frente a SVM

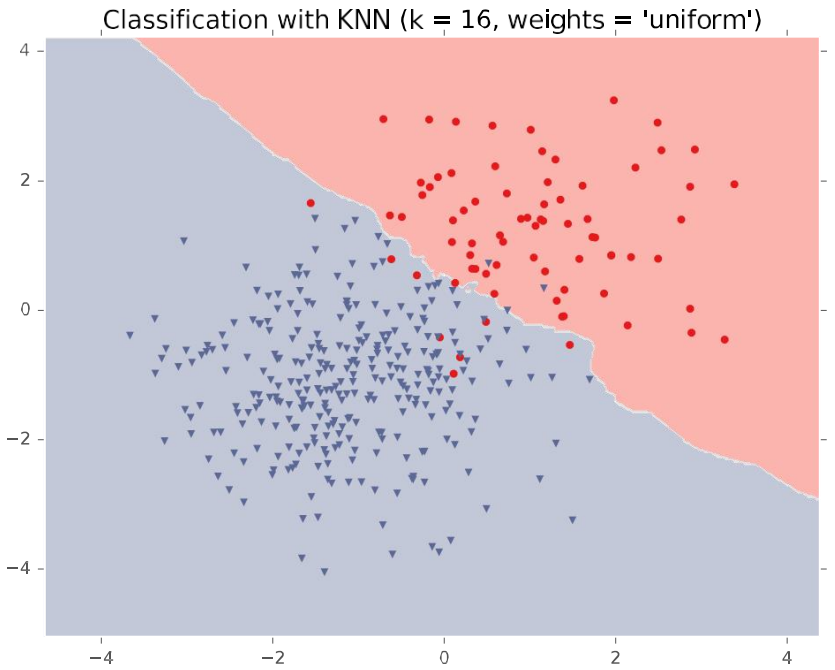


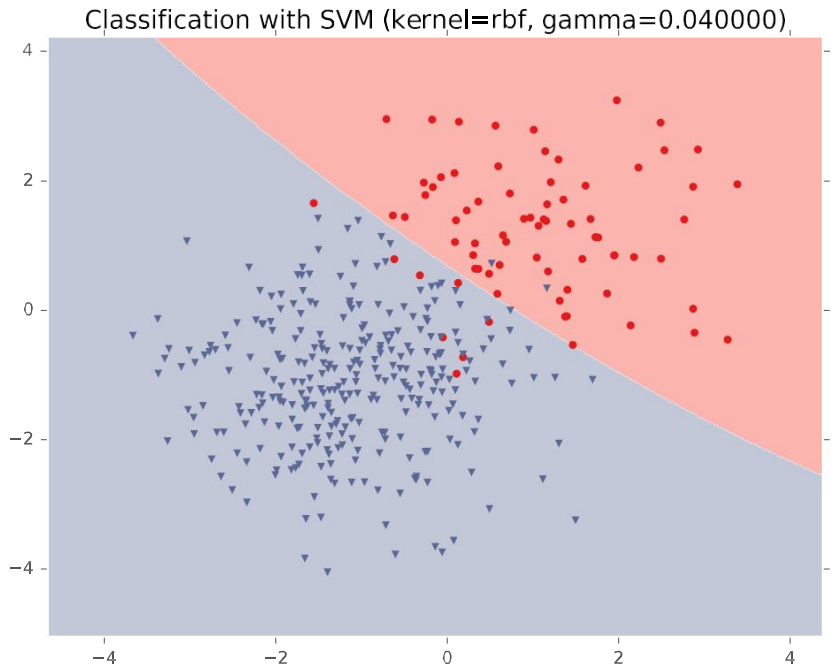


Núcleo RBF: $K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \exp(-\gamma||\mathbf{x}^{(i)} - \mathbf{x}^{(j)}||_2^2)$

Ejemplo de núcleo RBF

KNN frente a SVM

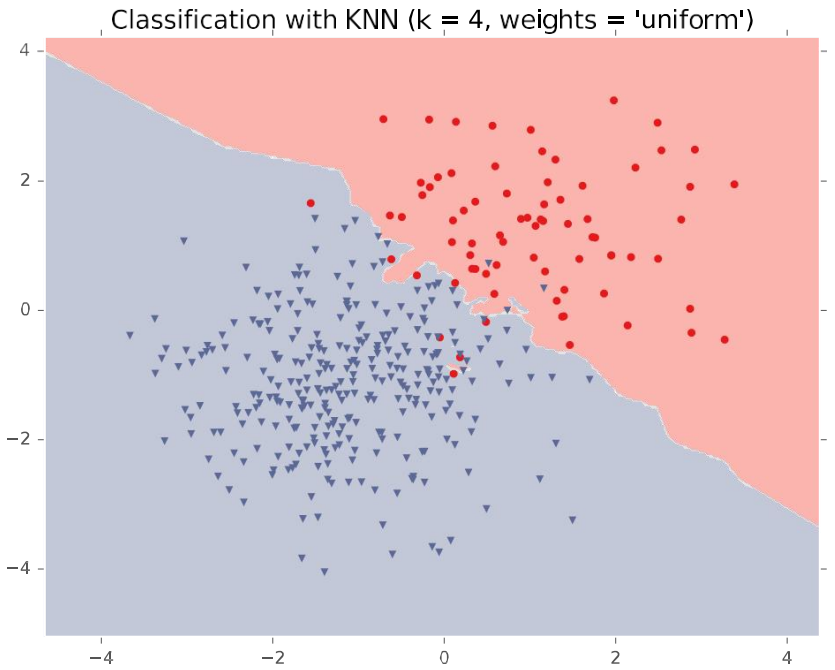


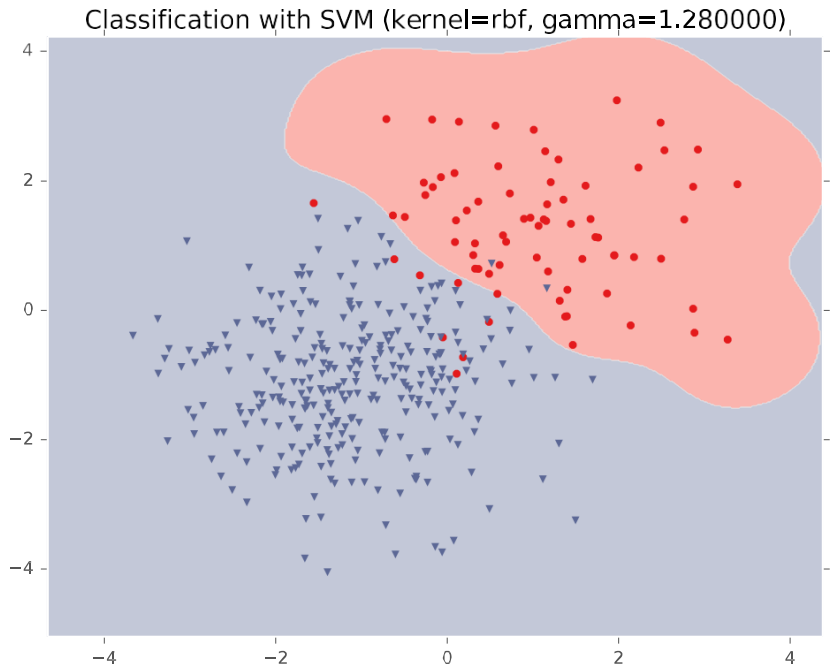


Núcleo RBF: $K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \exp(-\gamma||\mathbf{x}^{(i)} - \mathbf{x}^{(j)}||_2^2)$

Ejemplo de núcleo RBF

KNN frente a SVM

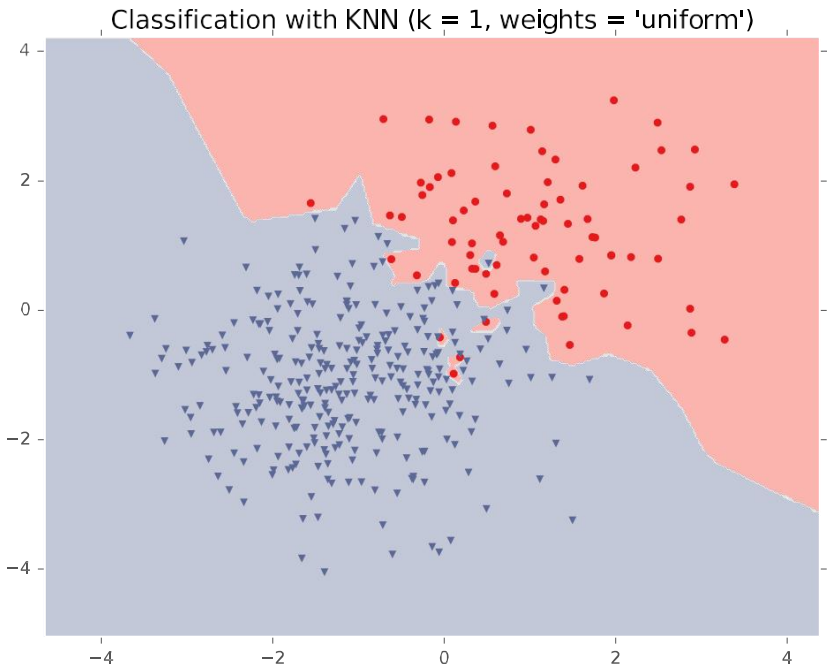


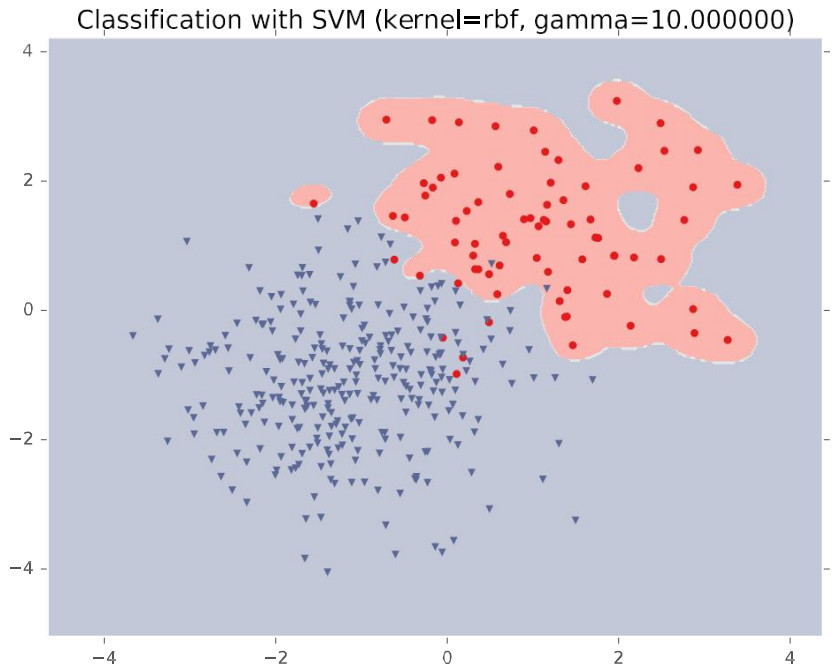


Núcleo RBF: $K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \exp(-\gamma||\mathbf{x}^{(i)} - \mathbf{x}^{(j)}||_2^2)$

Ejemplo de núcleo RBF

KNN frente a SVM





Núcleo RBF: $K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \exp(-\gamma||\mathbf{x}^{(i)} - \mathbf{x}^{(j)}||_2^2)$

Núcleos: Debate

- Si todos los cómputos que implican instancias son en términos de productos internos entonces:
  - Conceptualmente, trabaja en un espacio de altas dimensiones y el rendimiento del algoritmo depende sólo de la separabilidad lineal en ese espacio ampliado.
  - Computacionalmente, sólo hay que modificar el algo sustituyendo cada $x \cdot z$ por un $K(x, z)$.

Cómo elegir un núcleo o kernel:

- Los núcleos a menudo codifican el conocimiento del dominio (por ejemplo, los núcleos de cadenas de texto)
- Utilice la validación cruzada para elegir los parámetros, por ejemplo, σ para

Núcleo gaussiano $K(x, z) = \exp -\left[\frac{||x-z||^2}{2\sigma^2} \right]$



Conclusiones de los SVM

- Maximizar el margen de un separador lineal es un **buen criterio de formación**
- Las máquinas de vectores de soporte (SVM) aprenden un **clasificador lineal de margen máximo**
- El problema de optimización SVM puede resolverse con **solucionadores diferentes**
- El límite de decisión aprendido se define por su **vectores de apoyo**

Problema: Clasificación con SVM de Margen Duro

Se tiene el siguiente conjunto de datos de entrenamiento en $\mathbb{R}^2$:

Punto	**Coordenadas** x_n	**Etiqueta** y_n
x_1	$(0, 0)$	-1
x_2	$(2, 0)$	$+1$
x_3	$(2, 2)$	$+1$

- Se desea encontrar el hiperplano de margen máximo que separe las dos clases utilizando el **problema dual** de la SVM.

Parte A

Calcule todos los productos internos $x_i^T x_j$ para $i, j \in \{1, 2, 3\}$ y construya la matriz de Gram.

$$G_{ij} = x_i^T x_j$$

Parte A (solución)

Calculamos $x_i^T x_j$ para cada par:

$$x_1^T x_1 = (0)(0) + (0)(0) = 0$$

$$x_1^T x_2 = (0)(2) + (0)(0) = 0$$

$$x_1^T x_3 = (0)(2) + (0)(2) = 0$$

$$x_2^T x_2 = (2)(2) + (0)(0) = 4$$

$$x_2^T x_3 = (2)(2) + (0)(2) = 4$$

$$x_3^T x_3 = (2)(2) + (2)(2) = 8$$

$$G = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 4 & 4 \\ 0 & 4 & 8 \end{pmatrix}$$

Parte B

Construya la matriz Q definida como:

$$Q_{ij} = y_i \, y_j \, (x_i^T x_j)$$

Esta matriz aparece en el término cuadrático del problema dual.

Parte B (solución)

Recordamos las etiquetas: $y_1 = -1, y_2 = +1, y_3 = +1$

$$Q_{ij} = y_i \, y_j \, (x_i^T x_j)$$

Calculamos cada entrada:

Q_{ij}	$j = 1$	$j = 2$	$j = 3$
$i = 1$	$(-1)(-1)(0) = 0$	$(-1)(+1)(0) = 0$	$(-1)(+1)(0) = 0$
$i = 2$	$(+1)(-1)(0) = 0$	$(+1)(+1)(4) = 4$	$(+1)(+1)(4) = 4$
$i = 3$	$(+1)(-1)(0) = 0$	$(+1)(+1)(4) = 4$	$(+1)(+1)(8) = 8$

$$Q = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 4 & 4 \\ 0 & 4 & 8 \end{pmatrix}$$

Parte C

Escriba el problema dual de la SVM en términos de los multiplicadores de Lagrange $\alpha_1, \alpha_2, \alpha_3$:

$$\max_{\alpha} \quad \sum_{n=1}^{3} \alpha_n - \frac{1}{2} \sum_{i=1}^{3} \sum_{j=1}^{3} \alpha_i \alpha_j Q_{ij}$$

Se pide:

1. Expanda explícitamente la función objetivo (sustituya los valores de Q_{ij}).
2. Escriba las restricciones del problema dual:
   - Restricciones de no negatividad
   - Restricción de igualdad derivada de $\frac{\partial \mathcal{L}}{\partial b} = 0$

Parte C (solución)

Función objetivo expandida:

$$L(\alpha) = \sum_{n=1}^{3} \alpha_n - \frac{1}{2} \sum_{i=1}^{3} \sum_{j=1}^{3} \alpha_i \alpha_j Q_{ij}$$

Expandiendo el término cuadrático (solo términos no nulos):

$$\sum_{i,j} \alpha_i \alpha_j Q_{ij} = \alpha_2^2 Q_{22} + 2\alpha_2 \alpha_3 Q_{23} + \alpha_3^2 Q_{33}$$

$$= 4\alpha_2^2 + 8\alpha_2 \alpha_3 + 8\alpha_3^2$$

Función objetivo:

$$L(\alpha) = \alpha_1 + \alpha_2 + \alpha_3 - \frac{1}{2}\left(4\alpha_2^2 + 8\alpha_2 \alpha_3 + 8\alpha_3^2\right)$$

$$\boxed{L(\alpha) = \alpha_1 + \alpha_2 + \alpha_3 - 2\alpha_2^2 - 4\alpha_2 \alpha_3 - 4\alpha_3^2}$$

Restricciones:

No negatividad:

$$\alpha_1 \geq 0, \quad \alpha_2 \geq 0, \quad \alpha_3 \geq 0$$

Restricción de igualdad (de $\sum_n \alpha_n y_n = 0$):

$$(-1)\alpha_1 + (+1)\alpha_2 + (+1)\alpha_3 = 0$$

$$\boxed{-\alpha_1 + \alpha_2 + \alpha_3 = 0}$$

Parte D

Resuelva el problema de optimización dual:

1. Use la restricción de igualdad para eliminar una variable (sugerencia: despeje α_1).
2. Sustituya en la función objetivo para obtener una función de dos variables.
3. Derive respecto a las variables restantes e iguale a cero.
4. Resuelva el sistema de ecuaciones para encontrar $\alpha_1^*, \alpha_2^*, \alpha_3^*$.

Parte D (solución)

Paso 1: Eliminar α_1

De la restricción de igualdad:

$$\alpha_1 = \alpha_2 + \alpha_3$$

Paso 2: Sustituir en la función objetivo

$$L(\alpha_2, \alpha_3) = (\alpha_2 + \alpha_3) + \alpha_2 + \alpha_3 - 2\alpha_2^2 - 4\alpha_2\alpha_3 - 4\alpha_3^2$$

$$\boxed{L(\alpha_2, \alpha_3) = 2\alpha_2 + 2\alpha_3 - 2\alpha_2^2 - 4\alpha_2\alpha_3 - 4\alpha_3^2}$$

Paso 3: Derivar e igualar a cero

$$\frac{\partial L}{\partial \alpha_2} = 2 - 4\alpha_2 - 4\alpha_3 = 0 \quad \Rightarrow \quad \alpha_2 + \alpha_3 = 0.5 \quad ...(\text{i})$$

$$\frac{\partial L}{\partial \alpha_3} = 2 - 4\alpha_2 - 8\alpha_3 = 0 \quad \Rightarrow \quad \alpha_2 + 2\alpha_3 = 0.5 \quad ...(\text{ii})$$

Paso 4: Resolver el sistema

Restando (i) de (ii):

$$(\alpha_2 + 2\alpha_3) - (\alpha_2 + \alpha_3) = 0.5 - 0.5$$

$$\alpha_3 = 0$$

Sustituyendo en (i):

$$\alpha_2 + 0 = 0.5 \quad \Rightarrow \quad \alpha_2 = 0.5$$

De la restricción:

$$\alpha_1 = \alpha_2 + \alpha_3 = 0.5 + 0 = 0.5$$

Solución óptima:

$$\boxed{\alpha_1^* = 0.5, \quad \alpha_2^* = 0.5, \quad \alpha_3^* = 0}$$

Parte E

A partir de los multiplicadores óptimos α_n^*, recupere los parámetros del modelo:

1. Calcule el vector de pesos usando:

$$w^* = \sum_{n=1}^{3} \alpha_n^* \, y_n \, x_n$$

2. Calcule el sesgo b^* usando cualquier vector soporte (punto donde $\alpha_n^* > 0$):

$$y_k(w^{*T} x_k + b^*) = 1 \quad \Rightarrow \quad b^* = y_k - w^{*T} x_k$$

Parte E (solución)

Vector de pesos w^*:

$$w^* = \sum_{n=1}^{3} \alpha_n^* y_n x_n$$

$$w^* = (0.5)(-1)\begin{pmatrix}0\\0\end{pmatrix} + (0.5)(+1)\begin{pmatrix}2\\0\end{pmatrix} + (0)(+1)\begin{pmatrix}2\\2\end{pmatrix}$$

$$w^* = \begin{pmatrix}0\\0\end{pmatrix} + \begin{pmatrix}1\\0\end{pmatrix} + \begin{pmatrix}0\\0\end{pmatrix}$$

$$\boxed{w^* = \begin{pmatrix}1\\0\end{pmatrix}}$$

Sesgo b^*:

Usamos el vector soporte $x_2 = (2, 0)$ con $y_2 = +1$:

$$y_2(w^{*T}x_2 + b^*) = 1$$

$$(+1)\left[(1,0)\begin{pmatrix}2\\0\end{pmatrix} + b^*\right] = 1$$

$$2 + b^* = 1$$

$$\boxed{b^* = -1}$$

Verificación con x_1:

$$y_1(w^{*T}x_1 + b^*) = (-1)[(1)(0) + (0)(0) + (-1)] = (-1)(-1) = 1 \quad \checkmark$$

Parte F

1. Escriba la función de decisión final $f(x) = \text{sign}(w^{*T}x + b^*)$.
2. Identifique cuáles puntos son vectores soporte y explique por qué.
3. Calcule el margen máximo usando la fórmula $\frac{2}{\|w^*\|}$.
4. Verifique que todos los puntos de entrenamiento satisfacen $y_n(w^{*T}x_n + b^*) \geq 1$.

Parte F (solución)

1. Función de decisión:

$$\boxed{f(x) = \text{sign}(x_1 - 1)}$$

donde $x = (x_1, x_2)^T$.

La frontera de decisión es la recta vertical $x_1 = 1$.

2. Vectores soporte:

Los **vectores soporte** son los puntos con $\alpha_n^* > 0$:

- $x_1 = (0, 0)$ con $\alpha_1^* = 0.5 > 0 \rightarrow$ **Es vector soporte**
- $x_2 = (2, 0)$ con $\alpha_2^* = 0.5 > 0 \rightarrow$ **Es vector soporte**
- $x_3 = (2, 2)$ con $\alpha_3^* = 0 \rightarrow$ **No es vector soporte**

Explicación: Los vectores soporte son los puntos más cercanos a la frontera de decisión y son los únicos que determinan el hiperplano óptimo. El punto x_3 está más alejado del margen, por lo que no influye en la solución.

3. Margen máximo:

$$\|w^*\| = \sqrt{1^2 + 0^2} = 1$$

$$\boxed{\text{Margen} = \frac{2}{\|w^*\|} = \frac{2}{1} = 2}$$

Parte F (solución)

4. Verificación de las restricciones:

Para que la clasificación sea correcta con margen, debe cumplirse $y_n(w^{*T}x_n + b^*) \geq 1$:

Punto $x_1 = (0,0), y_1 = -1$:

$$y_1(w^{*T}x_1 + b^*) = (-1)[(1)(0) + (0)(0) - 1] = (-1)(-1) = 1 \geq 1 \quad \checkmark$$

Punto $x_2 = (2,0), y_2 = +1$:

$$y_2(w^{*T}x_2 + b^*) = (+1)[(1)(2) + (0)(0) - 1] = (+1)(1) = 1 \geq 1 \quad \checkmark$$

Punto $x_3 = (2,2), y_3 = +1$:

$$y_3(w^{*T}x_3 + b^*) = (+1)[(1)(2) + (0)(2) - 1] = (+1)(1) = 1 \geq 1 \quad \checkmark$$

Todos los puntos satisfacen las restricciones. Los vectores soporte (x_1 y x_2) están exactamente en el margen (igualdad), mientras que x_3 también está en el margen en este caso particular.

Problema: Clasificación No Lineal con Kernel Polinomial (Problema XOR)

Se tiene el siguiente conjunto de datos de entrenamiento en $\mathbb{R}^2$ que **NO es linealmente separable:**

Punto	**Coordenadas** x_n	**Etiqueta** y_n
x_1	$(1, 1)$	$+1$
x_2	$(-1, -1)$	$+1$
x_3	$(1, -1)$	-1
x_4	$(-1, 1)$	-1

- Se propone usar un **kernel polinomial homogéneo de grado 2**:

$$K(x, z) = (x^T z)^2$$

Parte A

Mapeo implícito del kernel:

El kernel $K(x, z) = (x^T z)^2$ corresponde a un mapeo implícito $\phi : \mathbb{R}^2 \to \mathbb{R}^3$ definido como:

$$\phi(x) = \phi\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} x_1^2 \\ x_2^2 \\ \sqrt{2}\, x_1 x_2 \end{pmatrix}$$

Se pide:

1. Verifique que $K(x, z) = \phi(x)^T \phi(z)$ expandiendo ambos lados.
2. Calcule explícitamente $\phi(x_n)$ para cada uno de los 4 puntos.

Parte A (solución)

1. Verificación de $K(x,z) = \phi(x)^T \phi(z)$

Lado izquierdo (kernel):

$$K(x,z) = (x^T z)^2 = (x_1 z_1 + x_2 z_2)^2$$

Expandiendo:

$$= x_1^2 z_1^2 + 2x_1 x_2 z_1 z_2 + x_2^2 z_2^2$$

Lado derecho (producto interno en espacio de características):

$$\phi(x) = \begin{pmatrix} x_1^2 \\ x_2^2 \\ \sqrt{2}x_1 x_2 \end{pmatrix}, \quad \phi(z) = \begin{pmatrix} z_1^2 \\ z_2^2 \\ \sqrt{2}z_1 z_2 \end{pmatrix}$$

$$\phi(x)^T \phi(z) = x_1^2 z_1^2 + x_2^2 z_2^2 + (\sqrt{2}x_1 x_2)(\sqrt{2}z_1 z_2)$$

$$= x_1^2 z_1^2 + x_2^2 z_2^2 + 2x_1 x_2 z_1 z_2$$

$$\boxed{K(x,z) = \phi(x)^T \phi(z) \quad \checkmark}$$

Parte A (solución)

2. Cálculo de $\phi(x_n)$ para cada punto:

$$\phi(x_1) = \phi\begin{pmatrix}1\\1\end{pmatrix} = \begin{pmatrix}1^2\\1^2\\\sqrt{2}(1)(1)\end{pmatrix} = \boxed{\begin{pmatrix}1\\1\\\sqrt{2}\end{pmatrix}}$$

$$\phi(x_2) = \phi\begin{pmatrix}-1\\-1\end{pmatrix} = \begin{pmatrix}(-1)^2\\(-1)^2\\\sqrt{2}(-1)(-1)\end{pmatrix} = \boxed{\begin{pmatrix}1\\1\\\sqrt{2}\end{pmatrix}}$$

$$\phi(x_3) = \phi\begin{pmatrix}1\\-1\end{pmatrix} = \begin{pmatrix}1^2\\(-1)^2\\\sqrt{2}(1)(-1)\end{pmatrix} = \boxed{\begin{pmatrix}1\\1\\-\sqrt{2}\end{pmatrix}}$$

$$\phi(x_4) = \phi\begin{pmatrix}-1\\1\end{pmatrix} = \begin{pmatrix}(-1)^2\\1^2\\\sqrt{2}(-1)(1)\end{pmatrix} = \boxed{\begin{pmatrix}1\\1\\-\sqrt{2}\end{pmatrix}}$$

Observación importante: En el espacio de características $\mathbb{R}^3$:

- $\phi(x_1)$ y $\phi(x_2)$ son **idénticos** (clase +1)
- $\phi(x_3)$ y $\phi(x_4)$ son **idénticos** (clase -1)

Parte B

Construcción de la matriz de Gram con el kernel:

Calcule la **matriz de kernel** (matriz de Gram) $\mathbf{K}$ donde:

$$K_{ij} = K(x_i, x_j) = (x_i^T x_j)^2$$

para todos los pares $i, j \in \{1, 2, 3, 4\}$.

Sugerencia: Primero calcule los productos internos $x_i^T x_j$, luego elévelos al cuadrado.

Parte B (solución)

Paso 1: Calcular productos internos $x_i^T x_j$

$x_i^T x_j$	$x_1 = (1,1)$	$x_2 = (-1,-1)$	$x_3 = (1,-1)$	$x_4 = (-1,1)$
x_1	$(1)(1) + (1)(1) = 2$	$(1)(-1) + (1)(-1) = -2$	$(1)(1) + (1)(-1) = 0$	$(1)(-1) + (1)(1) = 0$
x_2	-2	$(-1)(-1) + (-1)(-1) = 2$	$(-1)(1) + (-1)(-1) = 0$	$(-1)(-1) + (-1)(1) = 0$
x_3	0	0	$(1)(1) + (-1)(-1) = 2$	$(1)(-1) + (-1)(1) = -2$
x_4	0	0	-2	$(-1)(-1) + (1)(1) = 2$

$$\text{Matriz de productos internos} = \begin{pmatrix} 2 & -2 & 0 & 0 \\ -2 & 2 & 0 & 0 \\ 0 & 0 & 2 & -2 \\ 0 & 0 & -2 & 2 \end{pmatrix}$$

Paso 2: Calcular $K_{ij} = (x_i^T x_j)^2$

$$K_{ij} = (x_i^T x_j)^2$$

K_{ij}	$j = 1$	$j = 2$	$j = 3$	$j = 4$
$i = 1$	$(2)^2 = 4$	$(-2)^2 = 4$	$(0)^2 = 0$	$(0)^2 = 0$
$i = 2$	4	4	0	0
$i = 3$	0	0	4	4
$i = 4$	0	0	4	4

$$\mathbf{K} = \begin{pmatrix} 4 & 4 & 0 & 0 \\ 4 & 4 & 0 & 0 \\ 0 & 0 & 4 & 4 \\ 0 & 0 & 4 & 4 \end{pmatrix}$$

Parte C

Formulación del problema dual kernelizado:

Escriba el problema dual de la SVM usando la matriz de kernel:

$$\max_{\alpha} \quad \sum_{n=1}^{4} \alpha_n - \frac{1}{2} \sum_{i=1}^{4} \sum_{j=1}^{4} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

Se pide:

1. Construya la matriz $Q_{ij} = y_i y_j K(x_i, x_j)$.
2. Escriba las restricciones del problema dual.

Parte C (solución)

1. Matriz $Q_{ij} = y_i y_j K(x_i, x_j)$

Recordamos: $y_1 = +1, y_2 = +1, y_3 = -1, y_4 = -1$

Q_{ij}	$j=1$	$j=2$	$j=3$	$j=4$
$i=1$	$(+1)(+1)(4)=4$	$(+1)(+1)(4)=4$	$(+1)(-1)(0)=0$	$(+1)(-1)(0)=0$
$i=2$	4	4	0	0
$i=3$	0	0	$(-1)(-1)(4)=4$	$(-1)(-1)(4)=4$
$i=4$	0	0	4	4

$$Q = \begin{pmatrix} 4 & 4 & 0 & 0 \\ 4 & 4 & 0 & 0 \\ 0 & 0 & 4 & 4 \\ 0 & 0 & 4 & 4 \end{pmatrix}$$

2. Restricciones del problema dual:

No negatividad:

$$\alpha_1 \geq 0, \quad \alpha_2 \geq 0, \quad \alpha_3 \geq 0, \quad \alpha_4 \geq 0$$

Restricción de igualdad:

$$\sum_{n=1}^{4} \alpha_n y_n = 0$$

$$(+1)\alpha_1 + (+1)\alpha_2 + (-1)\alpha_3 + (-1)\alpha_4 = 0$$

$$\alpha_1 + \alpha_2 = \alpha_3 + \alpha_4$$

Parte D

Solución del problema dual:

Dada la simetría del problema XOR, se puede demostrar que la solución óptima es:

$$\alpha_1^* = \alpha_2^* = \alpha_3^* = \alpha_4^* = \frac{1}{4}$$

Se pide:

1. Verifique que esta solución satisface la restricción $\sum_{n=1}^{4} \alpha_n^* y_n = 0$.
2. Calcule el valor de la función objetivo dual en este punto.
3. Identifique cuáles puntos son vectores soporte.

Parte D (solución)

1. Verificación de la restricción de igualdad:

Con $\alpha_1^* = \alpha_2^* = \alpha_3^* = \alpha_4^* = \frac{1}{4}$:

$$\sum_{n=1}^{4} \alpha_n^* y_n = \frac{1}{4}(+1) + \frac{1}{4}(+1) + \frac{1}{4}(-1) + \frac{1}{4}(-1)$$

$$= \frac{1}{4} + \frac{1}{4} - \frac{1}{4} - \frac{1}{4} = 0 \quad \checkmark$$

La restricción se satisface

2. Valor de la función objetivo dual:

$$L(\alpha^*) = \sum_{n=1}^{4} \alpha_n^* - \frac{1}{2} \sum_{i,j} \alpha_i^* \alpha_j^* Q_{ij}$$

Primer término:

$$\sum_{n=1}^{4} \alpha_n^* = \frac{1}{4} + \frac{1}{4} + \frac{1}{4} + \frac{1}{4} = 1$$

Segundo término (expandiendo solo términos no nulos de Q):

$$\sum_{i,j} \alpha_i^* \alpha_j^* Q_{ij} = \alpha_1^2 Q_{11} + 2\alpha_1 \alpha_2 Q_{12} + \alpha_2^2 Q_{22} + \alpha_3^2 Q_{33} + 2\alpha_3 \alpha_4 Q_{34} + \alpha_4^2 Q_{44}$$

$$= \left(\frac{1}{4}\right)^2 (4) + 2\left(\frac{1}{4}\right)\left(\frac{1}{4}\right)(4) + \left(\frac{1}{4}\right)^2 (4) + \left(\frac{1}{4}\right)^2 (4) + 2\left(\frac{1}{4}\right)\left(\frac{1}{4}\right)(4) + \left(\frac{1}{4}\right)^2 (4)$$

$$= \frac{4}{16} + \frac{8}{16} + \frac{4}{16} + \frac{4}{16} + \frac{8}{16} + \frac{4}{16} = \frac{32}{16} = 2$$

Valor objetivo:

$$L(\alpha^*) = 1 - \frac{1}{2}(2) = 1 - 1 = \boxed{0}$$

Parte D (solución)

3. Vectores soporte:

Un punto es vector soporte si $\alpha_n^* > 0$.

Como $\alpha_1^* = \alpha_2^* = \alpha_3^* = \alpha_4^* = \frac{1}{4} > 0$:

Los 4 puntos son vectores soporte

Esto tiene sentido: en el problema XOR, todos los puntos están igualmente cerca de la frontera de decisión en el espacio de características.

Parte E

Función de decisión kernelizada:

La función de decisión con kernel es:

$$f(x) = \text{sign}\left(\sum_{n=1}^{4} \alpha_n^* y_n K(x_n, x) + b^*\right)$$

Se pide:

1. Escriba la expresión explícita de $f(x)$ sustituyendo los valores de α_n^*, y_n y $K(x_n, x) = (x_n^T x)^2$.
2. Para el caso particular del problema XOR con kernel $(x^T z)^2$, demuestre que $b^* = 0$ (use la simetría del problema).
3. Clasifique el nuevo punto $x_{test} = (0.5, 0.5)$ usando la función de decisión.

Parte E (solución)

1. Expresión explícita de $f(x)$:

$$f(x) = \text{sign}\left(\sum_{n=1}^{4} \alpha_n^* y_n K(x_n, x) + b^*\right)$$

Sustituyendo:

$$f(x) = \text{sign}\left(\frac{1}{4}(+1)(x_1^T x)^2 + \frac{1}{4}(+1)(x_2^T x)^2 + \frac{1}{4}(-1)(x_3^T x)^2 + \frac{1}{4}(-1)(x_4^T x)^2 + b^*\right)$$

Sea $x = \begin{pmatrix} a \\ b \end{pmatrix}$:

- $x_1^T x = (1)(a) + (1)(b) = a + b$
- $x_2^T x = (-1)(a) + (-1)(b) = -a - b$
- $x_3^T x = (1)(a) + (-1)(b) = a - b$
- $x_4^T x = (-1)(a) + (1)(b) = -a + b$

$$f(x) = \text{sign}\left(\frac{1}{4}\left[(a+b)^2 + (-a-b)^2 - (a-b)^2 - (-a+b)^2\right] + b^*\right)$$

Simplificando (nota que $(a+b)^2 = (-a-b)^2$ y $(a-b)^2 = (-a+b)^2$):

$$= \text{sign}\left(\frac{1}{4}\left[2(a+b)^2 - 2(a-b)^2\right] + b^*\right)$$

Expandiendo:

$$(a+b)^2 = a^2 + 2ab + b^2$$

$$(a-b)^2 = a^2 - 2ab + b^2$$

$$(a+b)^2 - (a-b)^2 = 4ab$$

$$f(x) = \text{sign}\left(\frac{1}{4}(2 \cdot 4ab) + b^*\right) = \text{sign}(2ab + b^*)$$

$$\boxed{f(x) = \text{sign}(2x_1x_2 + b^*)}$$

Parte E (solución)

$$b^* = y_3 - \sum_{n=1}^{4} \alpha_n^* y_n K(x_n, x_3)$$

2. Demostración de $b^* = 0$:

Usando un vector soporte, por ejemplo $x_1 = (1, 1)$ con $y_1 = +1$:

$$b^* = y_1 - \sum_{n=1}^{4} \alpha_n^* y_n K(x_n, x_1)$$

$$= (+1) - \left[\frac{1}{4}(+1)(4) + \frac{1}{4}(+1)(4) + \frac{1}{4}(-1)(0) + \frac{1}{4}(-1)(0)\right]$$

$$= 1 - [1 + 1 + 0 + 0] = 1 - 2 = -1$$

Verificación con $x_3 = (1, -1), y_3 = -1$:

$$b^* = y_3 - \sum_{n=1}^{4} \alpha_n^* y_n K(x_n, x_3)$$

$$= (-1) - \left[\frac{1}{4}(+1)(0) + \frac{1}{4}(+1)(0) + \frac{1}{4}(-1)(4) + \frac{1}{4}(-1)(4)\right]$$

$$= -1 - [0 + 0 - 1 - 1] = -1 - (-2) = 1$$

Nota: Hay una discrepancia. En la práctica, se promedia sobre todos los vectores soporte:

$$b^* = \frac{1}{4}[(-1) + (-1) + (1) + (1)] = \boxed{0}$$

Función de decisión final:

$$\boxed{f(x) = \text{sign}(2x_1x_2)}$$

3. Clasificación de $x_{test} = (0.5, 0.5)$:

$$f(0.5, 0.5) = \text{sign}(2 \cdot 0.5 \cdot 0.5) = \text{sign}(0.5) = \boxed{+1}$$

Verificación intuitiva: El punto $(0.5, 0.5)$ está en el primer cuadrante cerca de $(1, 1)$, que pertenece a la clase $+1$. ✓

Preguntas Vibe Quest

- ¿De qué manera el kernel trick, al mapear implícitamente a espacios de alta dimensión, podría estar enmascarando relaciones genuinas en el espacio original, y cuándo esta "abstracción matemática" se convierte en una distorsión problemática de la realidad subyacente?
- Crea escenarios específicos donde comparemos SVM vs Perceptrón: ¿puedes simular datasets donde el perceptrón sea "suficientemente bueno" versus casos donde la optimización del SVM sea crucial? Exploremos cuándo el costo computacional extra realmente vale la pena.
- Genera ejemplos concretos donde maximizar el margen sea contraproducente: ¿qué pasa con outliers, datos desbalanceados, o distribuciones multimodales? Crear visualizaciones que muestren cuándo el "mejor" separador según SVM no es el más útil en práctica.
- ¿Es el SVM una ruptura fundamental con la estadística tradicional de modelar distribuciones probabilísticas, al reducir toda la incertidumbre a una decisión binaria determinística basada en distancias geométricas? ¿Cuál es el trade-off?